



Introduction to Computer and Programming Language

Objectives

- 1) To learn about the hardware and software components of computers
- 2) To learn about level of programming languages
- 3) To understand the activity of programming
- 4) To compile and run your first C program
- 5) To recognize 3 types of errors

WHAT IS A COMPUTER?

3



✓ An electronic device that stores and processes data

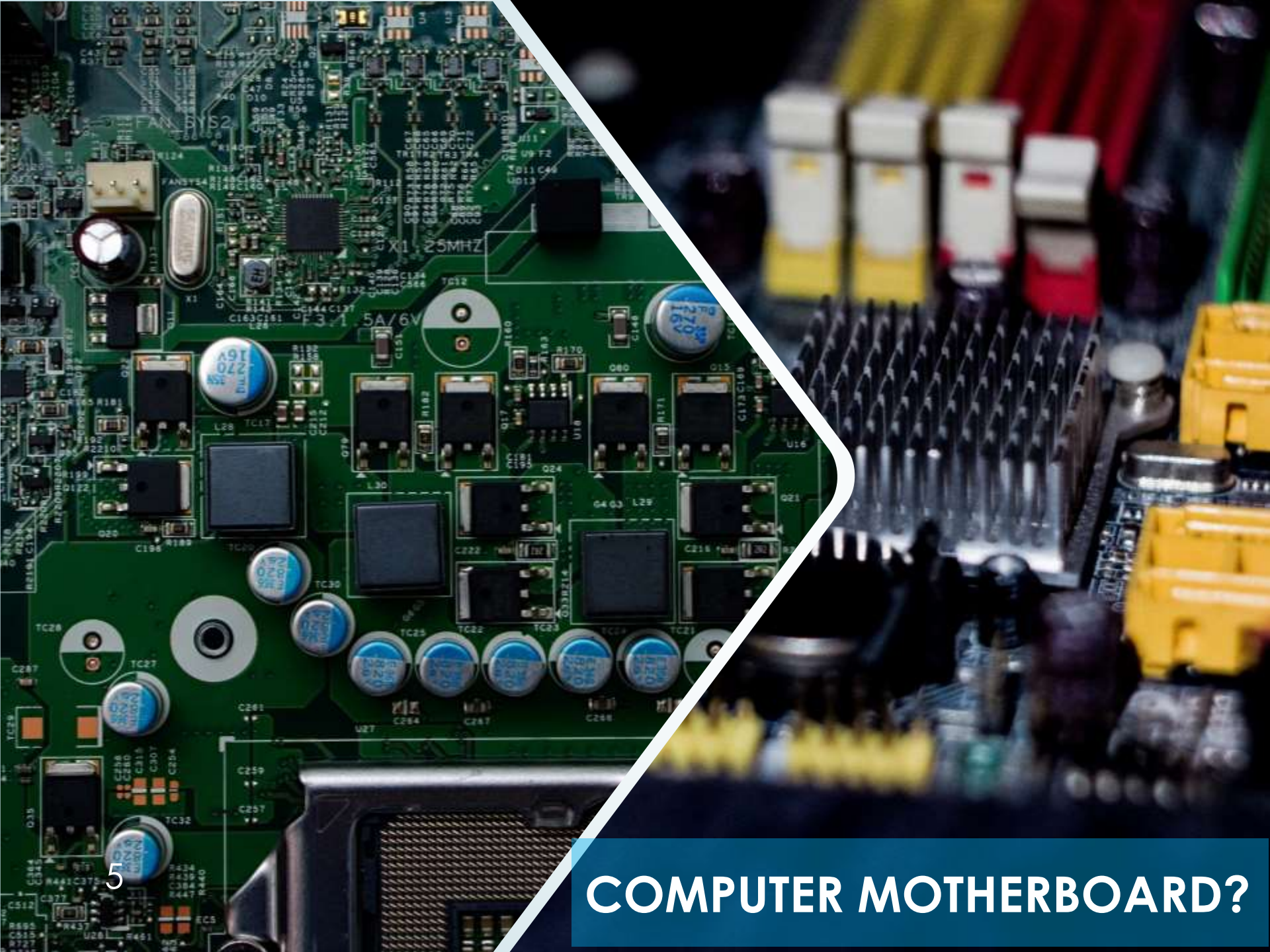
Calculator



$$10.50 + 12.00 = 22.50$$

$$50.00 - 22.50 = 27.50$$





5

COMPUTER MOTHERBOARD?

What is a computer?

tangible

- Consists of **hardware** and **software**.

Hardware

- the physical, tangible parts of a computer.
- keyboard, monitor, disks, wires, etc.

Software

Intangible

- programs (e.g. Ms Word) and data.
- Intangible parts of the computer.
- Both hardware and software form a tool that can be used to solve problems.

Computer System Organization

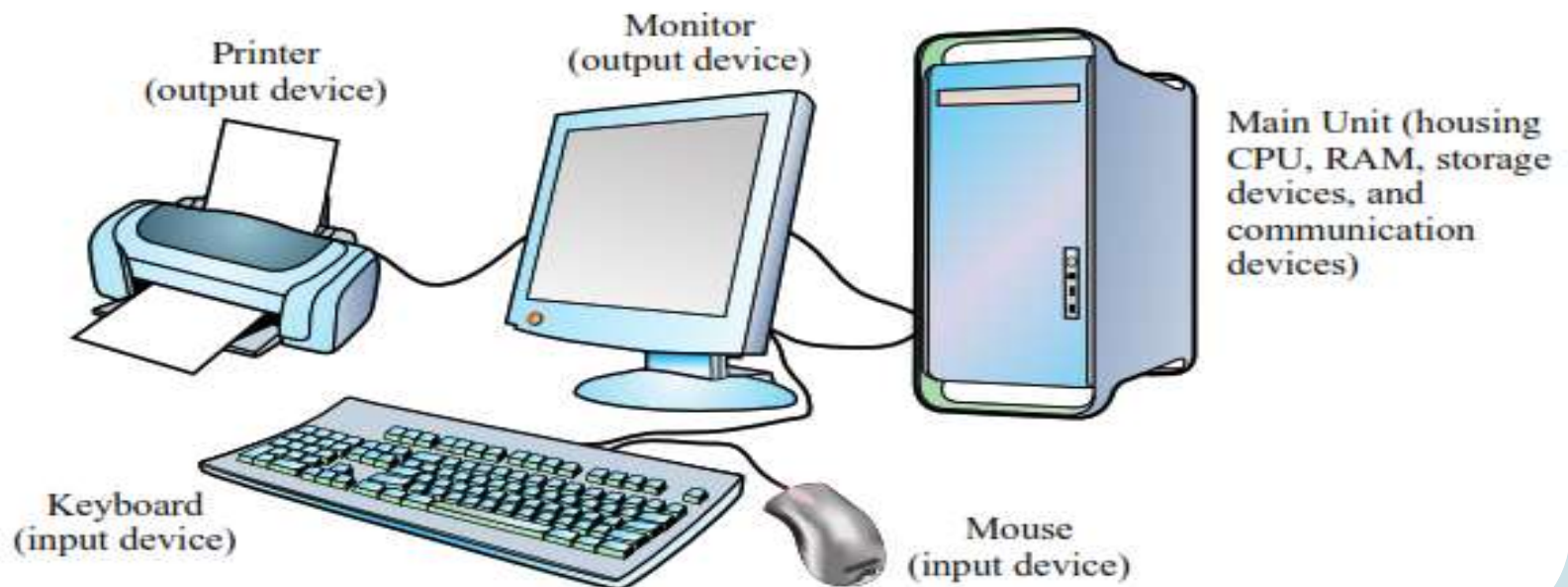
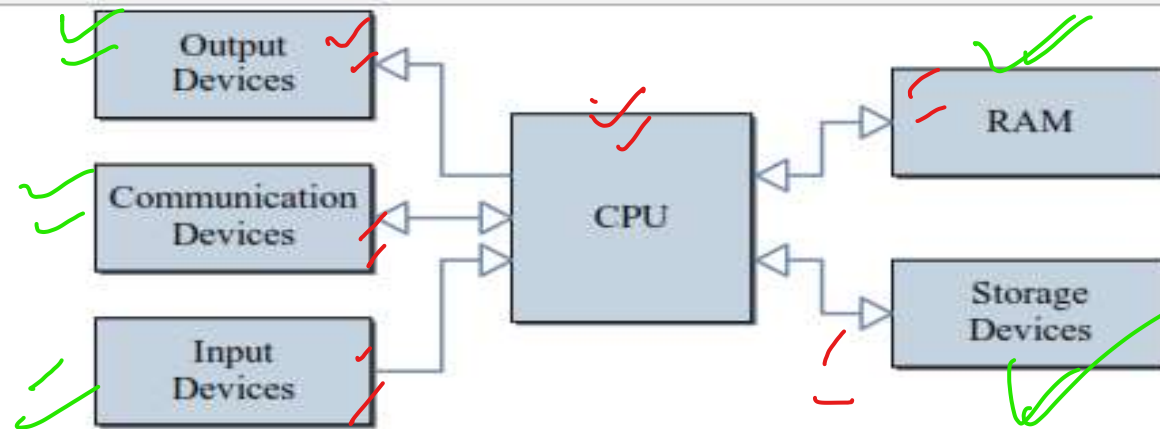
Computer System Organization

Hardware

- Central Processing Unit (CPU)
- Main Memory: Random Access Memory (RAM)
- Secondary Storage
- Input / Output Devices

Software

- System Software
- Application Software



Components of Computer System

Hardware Components of a Computer

CPU → brain of the computer

- Brain of the computer,
- ~~Handles all instructions~~ Handles all instructions that we give to the computer.
- ✓ Most expensive, the more expensive, the faster is the computer.

▪ CPU components:

- Control Unit ✓
- Program Counter ✓
- Registers ✓
- Arithmetic Logic Unit ✓
- Accumulator ✓

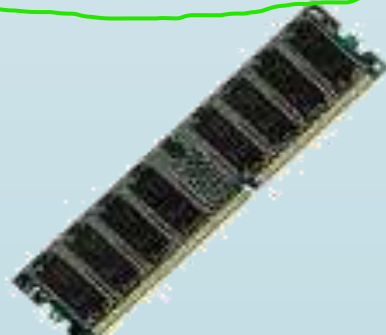


registers
ALU,
CU,
systems

Hardware Components of a Computer

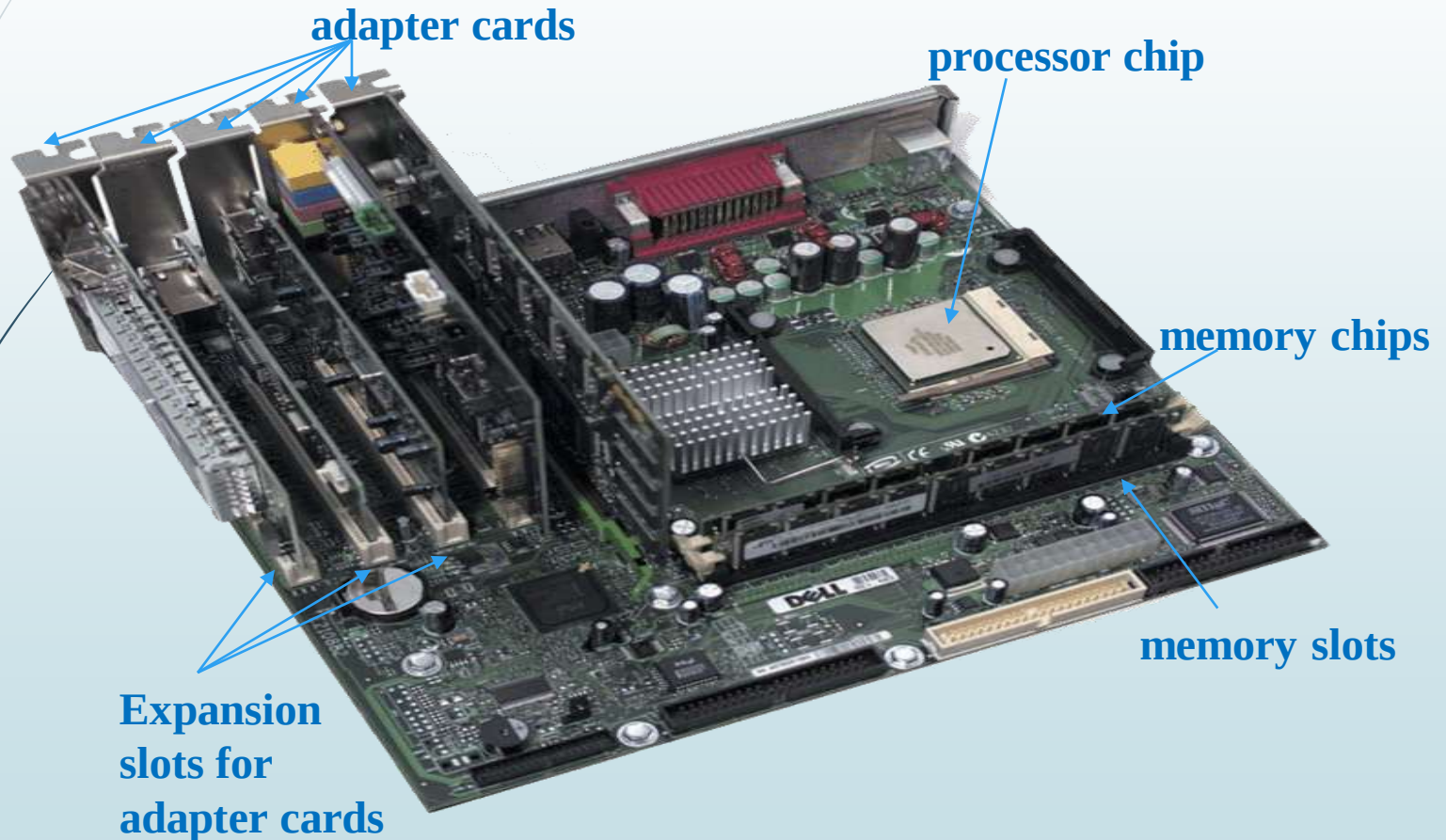
✓ RAM

- **Temporary** memory. ✓ Volatile
- Volatile (information stored in it will be lost when electric power supply is turned off).
- Directly connected to the CPU, using memory cells unit.



Hardware Components of a Computer:

MOTHERBOARD



Hardware Components of a Computer

SECONDARY STORAGE

- Provides **permanent** storage for information.
- Examples of secondary storage:
 - Hard disks
 - Floppy disks
 - CD-ROMs
 - USB Flash Drives

floppy disks
CD-ROMs
USB Flash drives

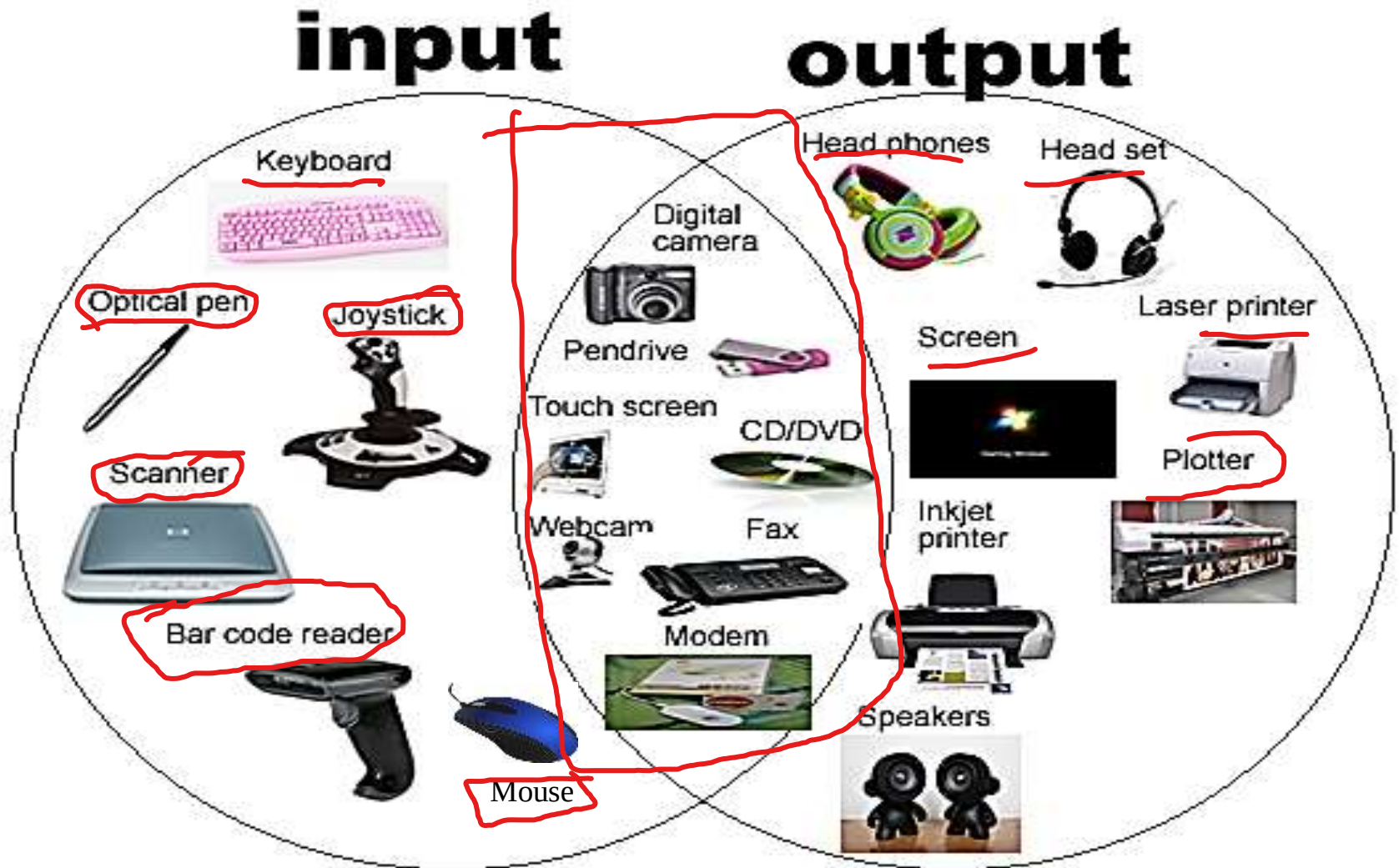


RAM VS. Secondary Storage

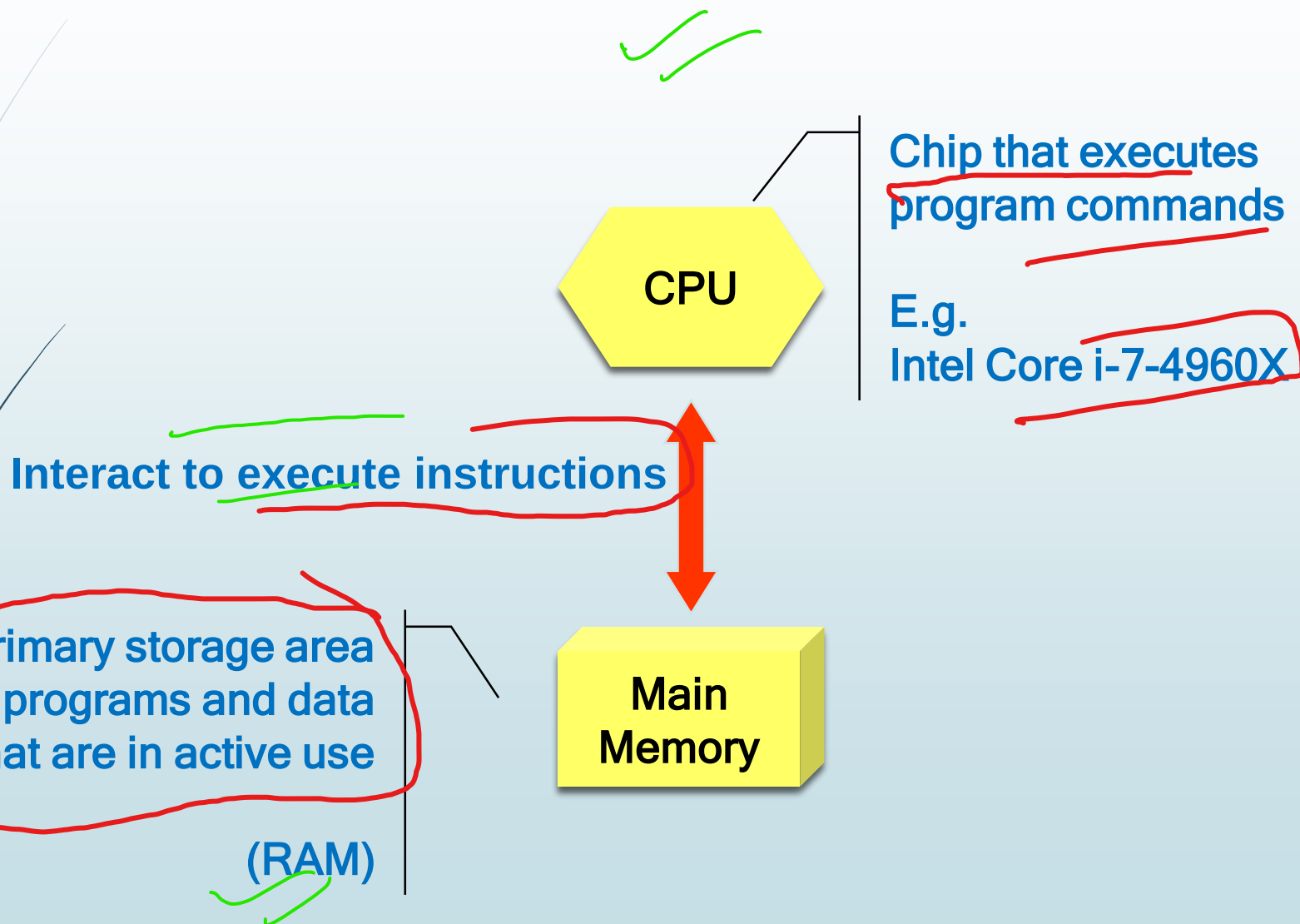
- **RAM / Main Memory / Primary Memory**
 - Volatile
 - Fast
 - Expensive
 - Low capacity
 - Works directly with the processor
- **Secondary Storage / Secondary Memory**
 - Nonvolatile
 - Slow
 - Cheap
 - Large capacity
 - Not connected directly to the processor

Hardware Components of a Computer

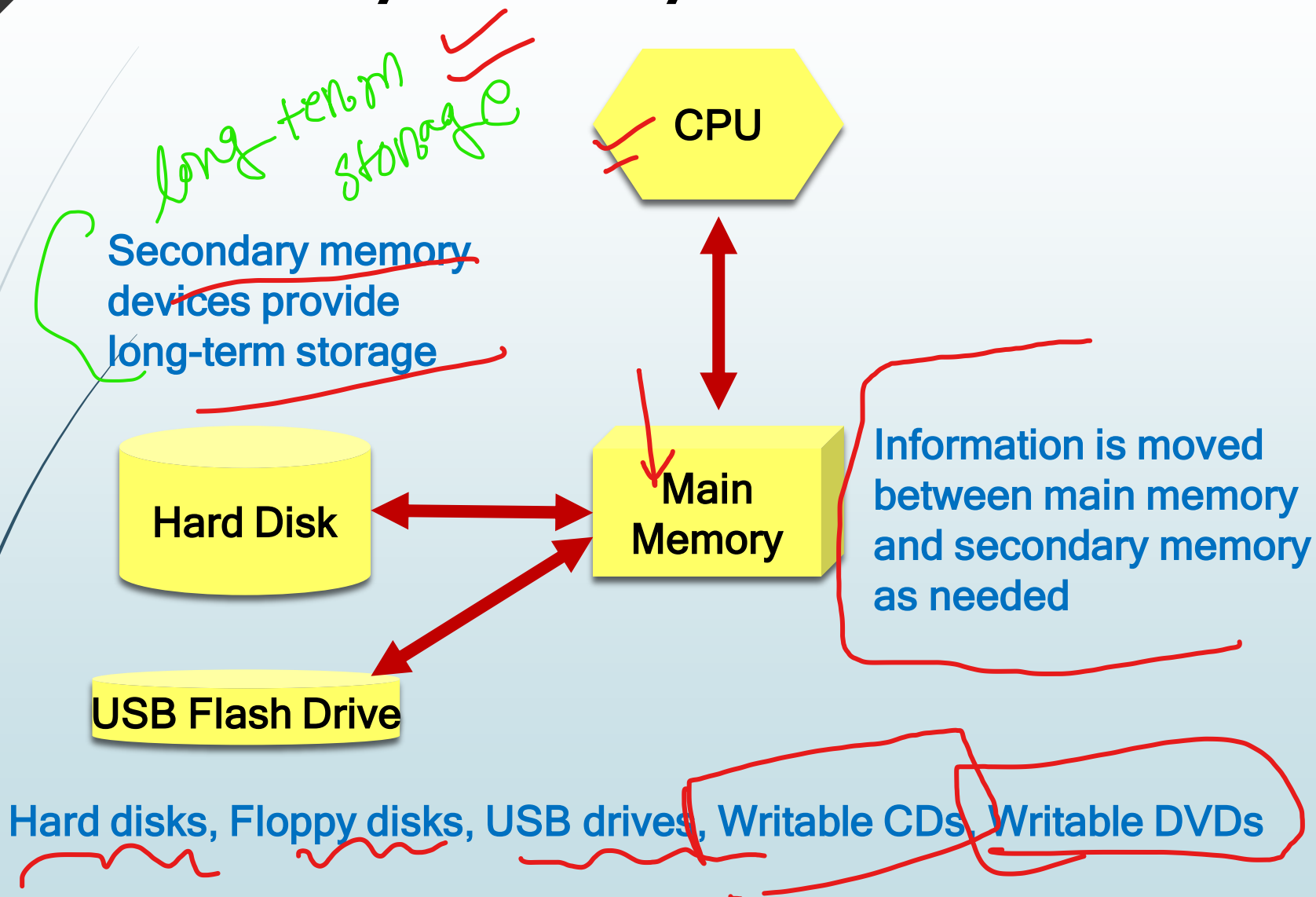
Allow human to interact with the computer



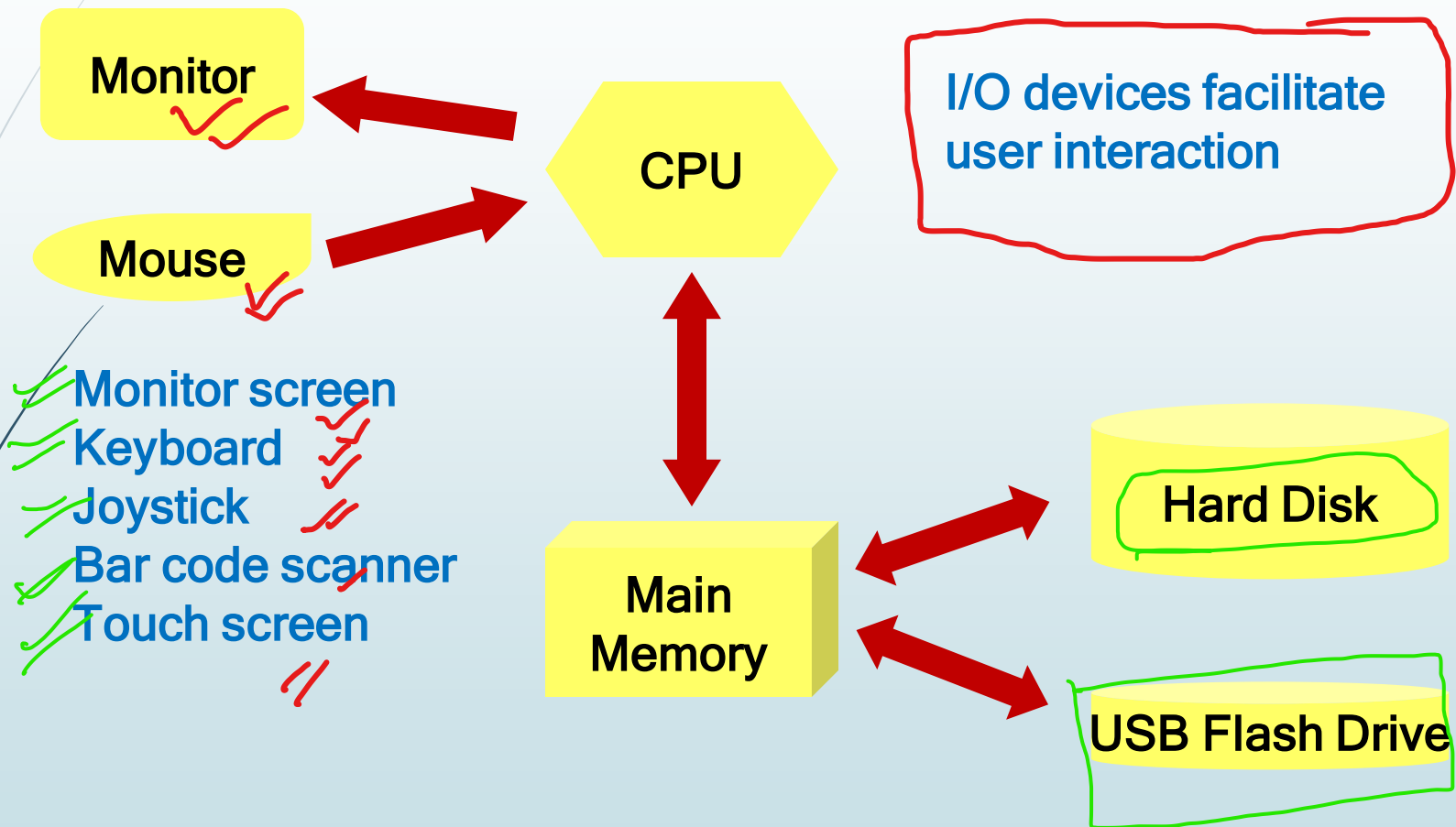
CPU and Main Memory



Secondary Memory Devices



Input / Output Devices



Think



Imagine that you are using a computer to write a letter.

- Where is the Microsoft Word program that you are running located?
- Where are the characters you have typed placed?

Software

→ sys
→ Application

- In contrast to hardware, software is an abstract, intangible entity.
- Software can be categorized as system or application software (refer next slide).
- It consists of program and data to be used to perform certain tasks.
- A program is a sequence of simple steps and operations, stated in a precise language that the hardware can interpret.
- The process of programming involve algorithm design & coding.

Software Categories

System Software

- System programs keep all the hardware and software running together smoothly.
- The most important system software is the operating system (OS).
- Controls all machine activities.
- Provides the user interface to the computer.
- Manages resources such as the CPU, memory & I/O.
- E.g: Windows 10, Unix, Linux, Mac OS.

Application Software

- Generic term for any other kind of software
- E.g: Ms Word processor, Excel Spreadsheets, Web browsers, games

Digital Information

- Computers store all information digitally:
 - numbers ✓
 - text ✓
 - graphics and images ✓
 - video ✓
 - audio ✓
 - program instructions ✓
- In some way, all information is digitized - broken down into pieces and represented as numbers ✓

0,1

Representing Text Digitally

- For example, every character is stored as a number, including spaces, digits, and punctuation.
- Corresponding upper and lower case letters are considered as different characters.

H i , H e a t h e r .

72 105 44 32 72 101 97 116 104 101 114 46

American Standard Code for Computer InterChange (ASCII)

23

The ASCII data set → 128 characters (0 until 127)
Character A in ASCII → 01000001

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	
	00 0000 0000	01 0000 0001	02 0000 0010	03 0000 0011	04 0000 0100	05 0000 0101	06 0000 0110	07 0000 0111	08 0000 1000	09 0000 1001	10 0000 1010	11 0000 1011	12 0000 1100	13 0000 1101	14 0000 1110	15 0000 1111	
0	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	HT	LF	VT	FF	CR	SO	SI	8
	□	┐	└	┘	↘	☒	✓	␣	↵	➤	≡	▼	⚡	⚡	⊗	⊙	
	16 0001 0000	17 0001 0001	18 0001 0010	19 0001 0011	20 0001 0100	21 0001 0101	22 0001 0110	23 0001 0111	24 0001 1000	25 0001 1001	26 0001 1010	27 0001 1011	28 0001 1100	29 0001 1101	30 0001 1110	31 0001 1111	
1	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US	9
	☐	⌚	⌚	⌚	⌚	✓	⌚	⌚	⌚	⌚	?	⌚	☐	☐	☐	☐	
	32 0010 0000	33 0010 0001	34 0010 0010	35 0010 0011	36 0010 0100	37 0010 0101	38 0010 0110	39 0010 0111	40 0010 1000	41 0010 1001	42 0010 1010	43 0010 1011	44 0010 1100	45 0010 1101	46 0010 1110	47 0010 1111	
2	SP	!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/	A
	48 0011 0000	49 0011 0001	50 0011 0010	51 0011 0011	52 0011 0100	53 0011 0101	54 0011 0110	55 0011 0111	56 0011 1000	57 0011 1001	58 0011 1010	59 0011 1011	60 0011 1100	61 0011 1101	62 0011 1110	63 0011 1111	
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?	B
	64 0100 0000	65 0100 0001	66 0100 0010	67 0100 0011	68 0100 0100	69 0100 0101	70 0100 0110	71 0100 0111	72 0100 1000	73 0100 1001	74 0100 1010	75 0100 1011	76 0100 1100	77 0100 1101	78 0100 1110	79 0100 1111	
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	C
	80 0101 0000	81 0101 0001	82 0101 0010	83 0101 0011	84 0101 0100	85 0101 0101	86 0101 0110	87 0101 0111	88 0101 1000	89 0101 1001	90 0101 1010	91 0101 1011	92 0101 1100	93 0101 1101	94 0101 1110	95 0101 1111	
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_	D
	96 0110 0000	97 0110 0001	98 0110 0010	99 0110 0011	100 0110 0100	101 0110 0101	102 0110 0110	103 0110 0111	104 0110 1000	105 0110 1001	106 0110 1010	107 0110 1011	108 0110 1100	109 0110 1101	110 0110 1110	111 0110 1111	
6	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	E
	112 0111 0000	113 0111 0001	114 0111 0010	115 0111 0011	116 0111 0100	117 0111 0101	118 0111 0110	119 0111 0111	120 0111 1000	121 0111 1001	122 0111 1010	123 0111 1011	124 0111 1100	125 0111 1101	126 0111 1110	127 0111 1111	
7	p	q	r	s	t	u	v	w	x	y	z	{		}	~	DEL	F

ASCII → American Standard Code for computer interchange

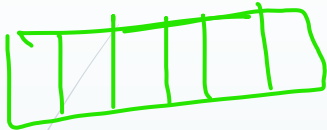
24

Binary Numbers

- Once information is digitized, it is represented and stored in memory using the binary number system.
- A single binary digit (0 or 1) is called a bit.
- A byte consists of 8 bits.
- Each byte in main memory resides at a numbered location called its address.

1 byte = 8 bits

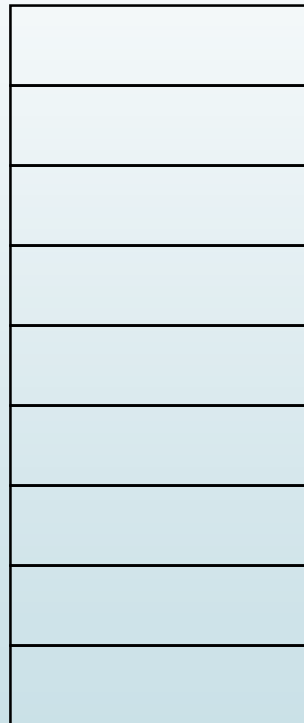
Main Memory



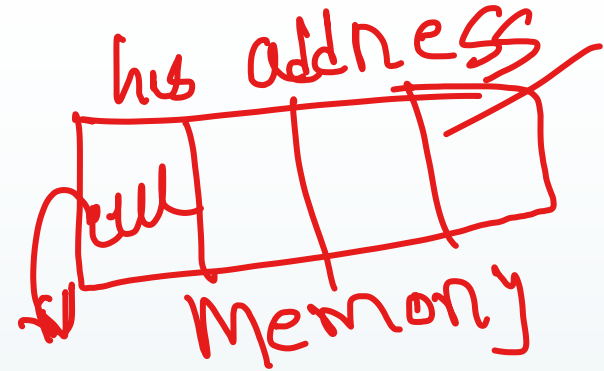
...

Addresses

9278
9279
9280
9281
9282
9283
9284
9285
9286



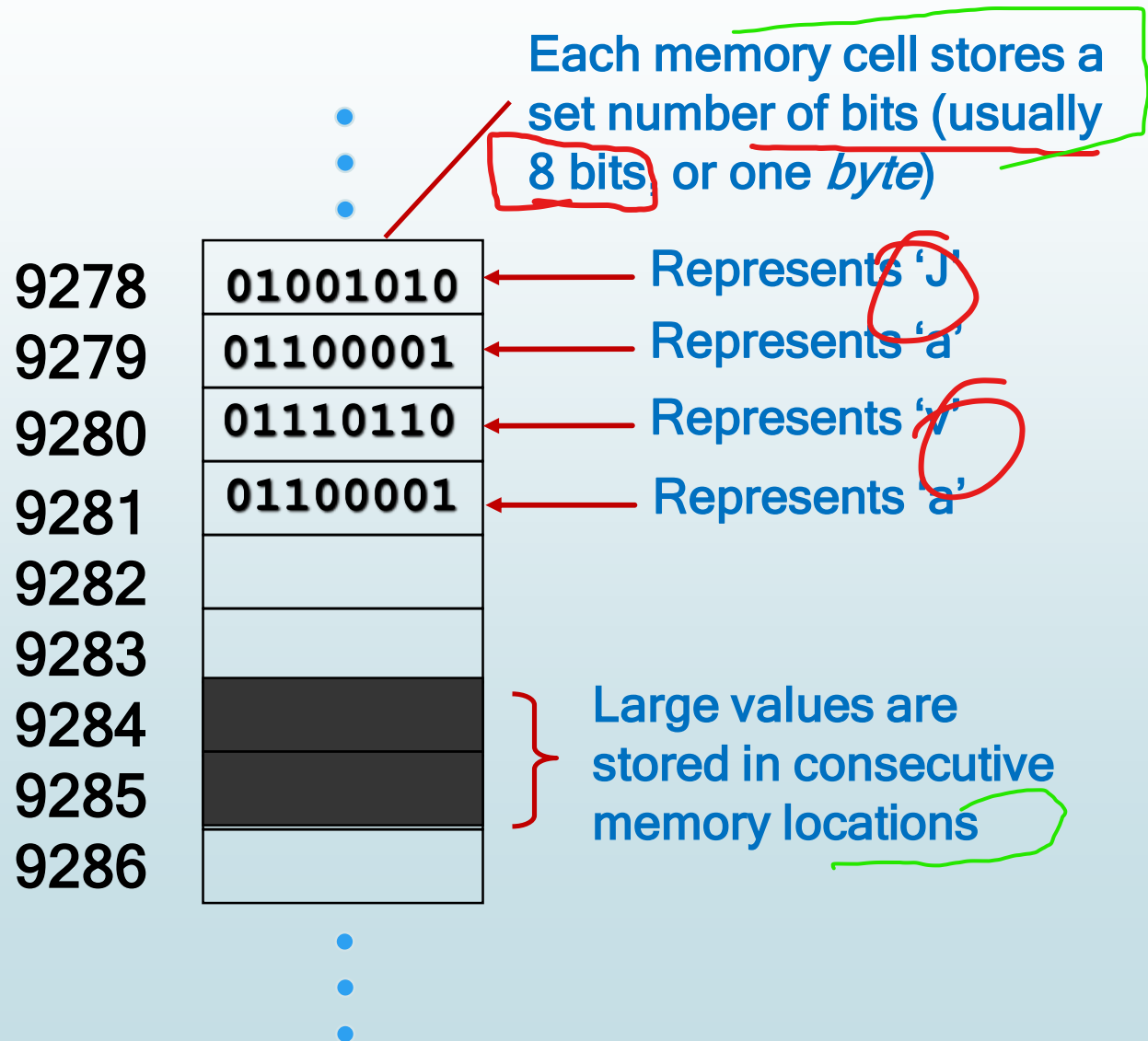
...



Main memory is divided into many memory locations (or cells)

Each memory cell has a numeric address, which uniquely identifies it

Storing Information



Main Memory

	Bytes	
•		
•		
•		
Byte addresses 3021	11110000	2-byte memory location at address 3021
3022	11001100	
3023	10101010	1-byte memory location at address 3023
3024	11001110	3-byte memory location at address 3024
3025	00110001	
3026	11100001	
3027	01100011	2-byte memory location at address 3027
3028	10100010	
3029	01111111	
3030	10000001	
3031	10111100	
•		
•		
•		

Storage Capacity

↑ Means number of bytes on HDD

- Every memory device has a **storage capacity**, indicating the number of bytes it can hold.
- Capacities are expressed in various units:

Unit	Symbol	Number of Bytes
kilobyte	KB	$2^{10} = 1024$
megabyte	MB	2^{20} (over 1 million)
gigabyte	GB	2^{30} (over 1 billion)
terabyte	TB	2^{40} (over 1 trillion)

KB
MB
GB
TB
 2^{10}
 2^{20}
 2^{30}
 2^{40}

↑

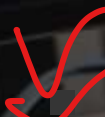
What is Programming?



Computer programming

- the art and science of designing and writing computer programs.

art of science of designing and writing computer programs



Computer program

- a sequence of instructions written in a programming language to achieve a task/to solve a problem.

HTML & JAVASCRIPT

<!DOCTYPE html>

<html>

<body>

<h2>What Can JavaScript Do?</h2>

<p>JavaScript can change HTML attribute values.</p>

<p>In this case JavaScript changes the value of the src (source) attribute of an image.</p>

<button

onclick="document.getElementById('myImage').src='pic_bulbon.gif'">

Turn on the light</button>

<button

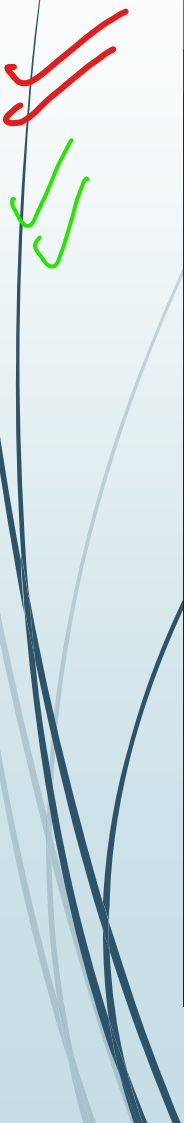
onclick="document.getElementById('myImage').src='pic_bulboff.gif'">

Turn off the light</button>

</body>

</html>

Java



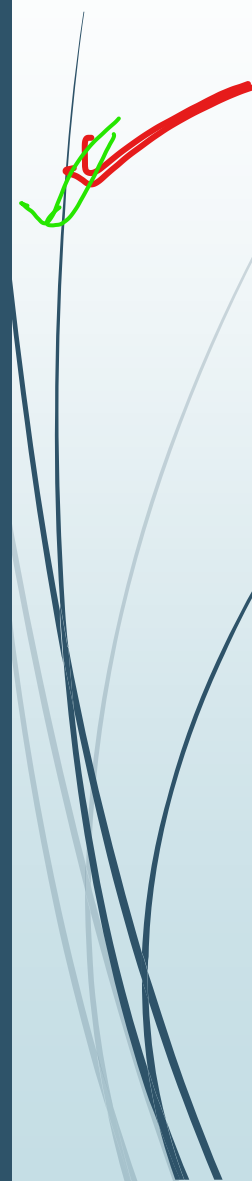
```
ExampleJava.java x
1 public class ExampleJava {
2     public static void main (String[] args) {
3         System.out.printf("\n\nThis is java\n\n");
4     }
5 }
```

```
Topic 1 — -bash — 80x24
[Noradilas-MacBook-Pro:Topic 1 adila$ ls
ExampleJava.java      example-cplus.cp
example-c.c           example-python.py
[Noradilas-MacBook-Pro:Topic 1 adila$ javac ExampleJava.java
[Noradilas-MacBook-Pro:Topic 1 adila$ ls
ExampleJava.class     example-c.c          example-python.py
ExampleJava.java      example-cplus.cp
[Noradilas-MacBook-Pro:Topic 1 adila$ java ExampleJava

This is java

Noradilas-MacBook-Pro:Topic 1 adila$
```

C



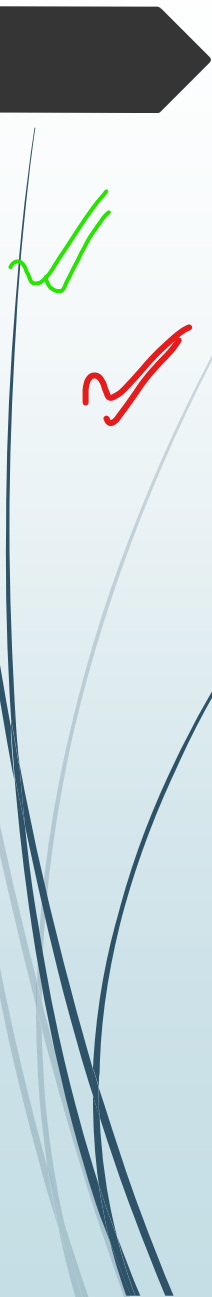
```
example-c.c x
1  #include <stdio.h>
2
3  int main() {
4      printf("\n\nThis is c programming\n\n");
5      return 0;
6  }
```

```
Topic 1 — -bash — 80x24
[Noradilas-MacBook-Pro:Topic 1 adila$ ls
ExampleJava.java      example-cplus.cp
example-c.c           example-python.py
[Noradilas-MacBook-Pro:Topic 1 adila$ gcc -o exampleC example-c.c
[Noradilas-MacBook-Pro:Topic 1 adila$ ls
ExampleJava.java      example-cplus.cp      exampleC
example-c.c           example-python.py
[Noradilas-MacBook-Pro:Topic 1 adila$ ./exampleC

This is c programming

Noradilas-MacBook-Pro:Topic 1 adila$
```


C++



```
example-cplus.cp x
1  #include <iostream>
2
3  int main() {
4      std::cout << "\n\nThis is c++\n\n";
5      return 0;
6  }
7

Topic 1 — -bash — 80x24
[Noradilas-MacBook-Pro:Topic 1 adila$ ls
ExampleJava.java      example-cplus.cp
example-c.c           example-python.py
[Noradilas-MacBook-Pro:Topic 1 adila$ g++ -o exampleCPlus example-cplus.cp
[Noradilas-MacBook-Pro:Topic 1 adila$ ls
ExampleJava.java      example-cplus.cp      exampleCPlus
example-c.c           example-python.py
[Noradilas-MacBook-Pro:Topic 1 adila$ ./exampleCPlus

This is c++

Noradilas-MacBook-Pro:Topic 1 adila$
```


Python

machine assembly with total language



```
example-python.py x
1 print('\n\nThis is python\n\n')
```

just print.

```
Topic 1 — -bash — 80x24
[Noradilas-MacBook-Pro:Topic 1 adila$ ls
ExampleJava.java      example-cplus.cp
example-c.c           example-python.py
[Noradilas-MacBook-Pro:Topic 1 adila$ python example-python.py

This is python

Noradilas-MacBook-Pro:Topic 1 adila$
```



Language Levels

(3 level)

- Levels of programming language levels:
 - **machine language** (computer directly understands)
 - **assembly language** (symbolic form, need minor translation)
 - **high-level language** (entire language need to be translated)
- Each type of CPU has its own specific machine language

Programming Languages

36

- Machine language

101101100110 011011010 ✓✓

Assembly ✓✓	Machine Code (Hex) ✓✓	Machine Code (Binary) ✓✓
MOV R0, #3	E3A00003	1110 0011 1010 0000 0000 0000 0000 0011
MOV R1, #4	E3A01004	1110 0011 1010 0001 0000 0000 0000 0100
ADD R2, R0, R1	E0802001	1110 0000 1000 0000 0010 0000 0000 0001

- Assembly language

MOV R0, #3 ; Load the immediate value 3 into register R0

MOV R1, #4 ; Load the immediate value 4 into register R1

ADD R2, R0, R1 ; Add the values in R0 and R1 and
store result in R2 (R2 = 3 + 4)

- High-level language

sum = 3+4;

Programming Languages

machine
assembly
high level
lang

- Each type of CPU executes instructions only in a particular **machine language**.

- A program must be translated into machine language before it can be executed.

- A **compiler** is a software tool which translates from high level language into a specific machine language. //

- An **assembler** translates from assembly language into a specific machine language.

✓ ✓ ✓
A compiler
(high level lang to machine lang)

→ high level lang

→ assembly

Programming Languages

Languages	Descriptions	Examples	Translators
Machine	Instruction in 0 and 1 bits	0111000011000001	None
Assembly	Instruction in mnemonic code	LOAD 3 STOR 4 ADD	Assembler Assembler
High-level	Similar to human language, e.g. <u>FORTRAN</u> , <u>COBOL</u> , <u>Pascal</u> , <u>C</u> , <u>C++</u> , <u>Java</u>	sum = 3 + 4; C, C++, Java Cobol, Fortran	Compiler & interpreter Compiler

The C Programming Language

- C is a general-purpose programming language created by Dennis Ritchie at the Bell Laboratories in 1972. ✓
- It is a very popular language, despite being old. The main reason for its popularity is because it is a fundamental language in the field of computer science.
- C is strongly associated with UNIX, as it was developed to write the UNIX operating system.

Unix operating system

C as imperative programming language

sequence of instructions

- An imperative language is a type of programming language in which the programmer explicitly tells the computer how to perform tasks.
- This means that the program is written as a sequence of detailed instructions, each step describing exactly what the computer should do to achieve the desired result.
- C is a classic example of an imperative programming language, which means it is designed to express computations as sequences of instructions that change a program's state.

imperative lang

C is considered an imperative language because it provides:

- ✓ Sequential execution of commands.
- ✓ Control structures for branching and looping.
- ✓ Direct manipulation of memory and program state.

Difference between C and C++

- C++ was **developed** as an extension of C, and both languages have almost the same syntax
- The main difference between C and C++ is that C++ support classes and objects, while C does not

C does not support class and objects.

✓✓ Get Started With C

To start using C, you need two things:

✓✱ A **text editor**, like Notepad, to write C code

✱ A **compiler**, like GCC, to translate the C code into a language that the computer will understand

C Install IDE

Visual Studio
Code::Blocks
~~Eclipse~~

- ✓ An IDE (Integrated Development Environment) is used to edit AND compile the code.
- Popular IDE's include Code::Blocks, Eclipse, and Visual Studio. These are all free, and they can be used to both edit and debug C code.
- Note: Web-based IDE's can work as well, but functionality is limited.
- You can find the latest version of Codeblocks at <http://www.codeblocks.org/>. Download the mingw-setup.exe file, which will install the text editor with a compiler.

C Quickstart

Code::Blocks

Visual Studio

Sublime

Eclipse

- ✓ Let's create our first C file.
- ✓ Open Codeblocks and go to File > New > Empty File.
- Write the following C code and save the file as myfirstprogram.c

myfirstprogram.c

```
#include <stdio.h>
```

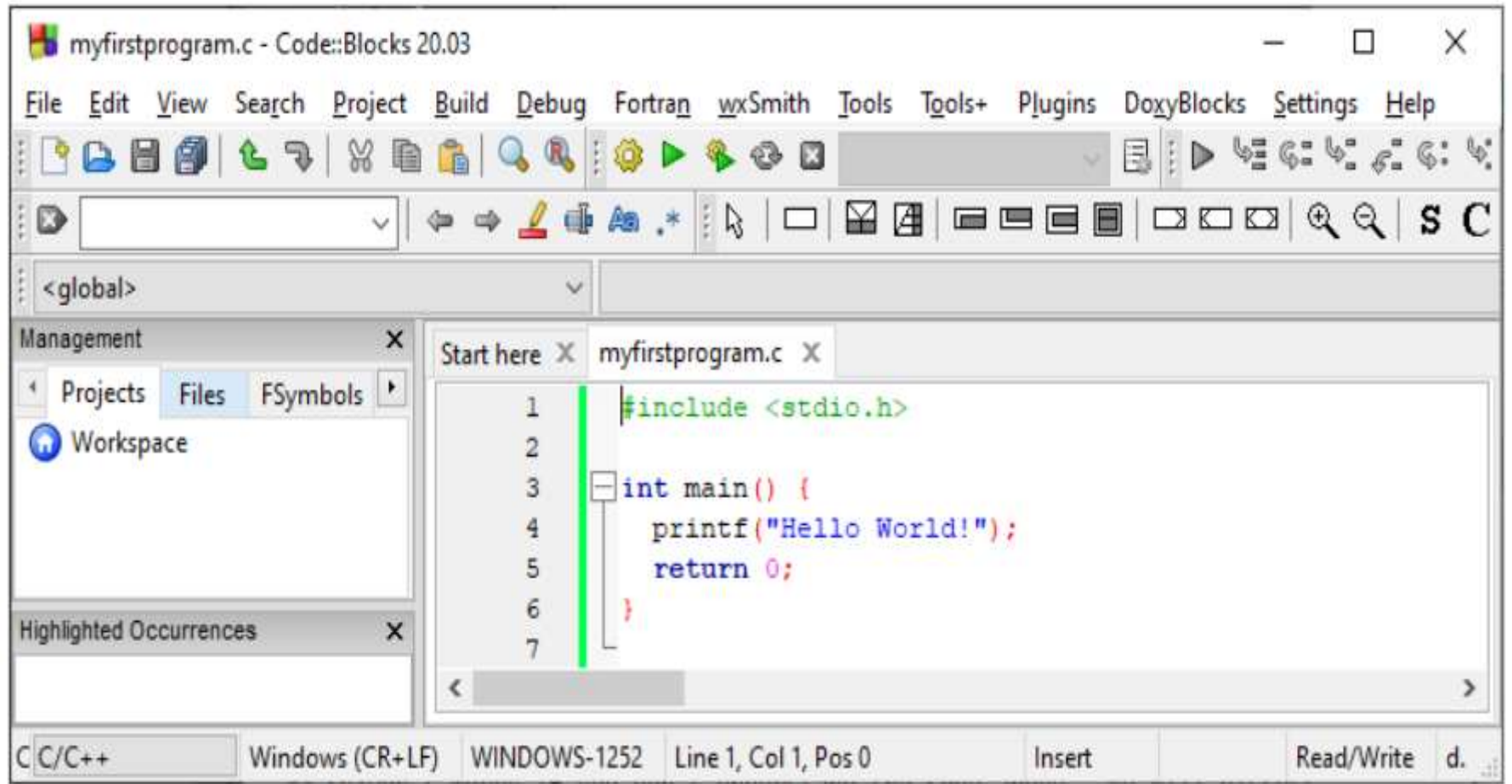
```
int main() {
```

```
    printf("Hello World!");
```

```
    return 0;
```

```
}
```


In Codeblocks, it should look like this



Compile and Run a program

- Go to Build > Build and Run to run (execute) the program.
- Output as below

```
Hello World!
```

```
Process returned 0 (0x0) execution time : 0.011 s
```

```
Press any key to continue.
```

C Translation

```
#include <stdio.h>
int main() {
    printf("Hello");
    return 0;
}
```

Errors

- It is common for programmer to make mistake in a program.
- Three kinds of errors:
 - **Syntax errors**
 - **Runtime errors**
 - **Logic errors**

Syntax
Runtime
Logic

Syntax Errors

grammatical

- Grammatical mistakes in a program.
- The grammatical rules for writing a program are very strict.
- The compiler catches syntax errors and prints an error message.
- Example: using a period where a program expects a semicolon

- `printf("Hello World!")`

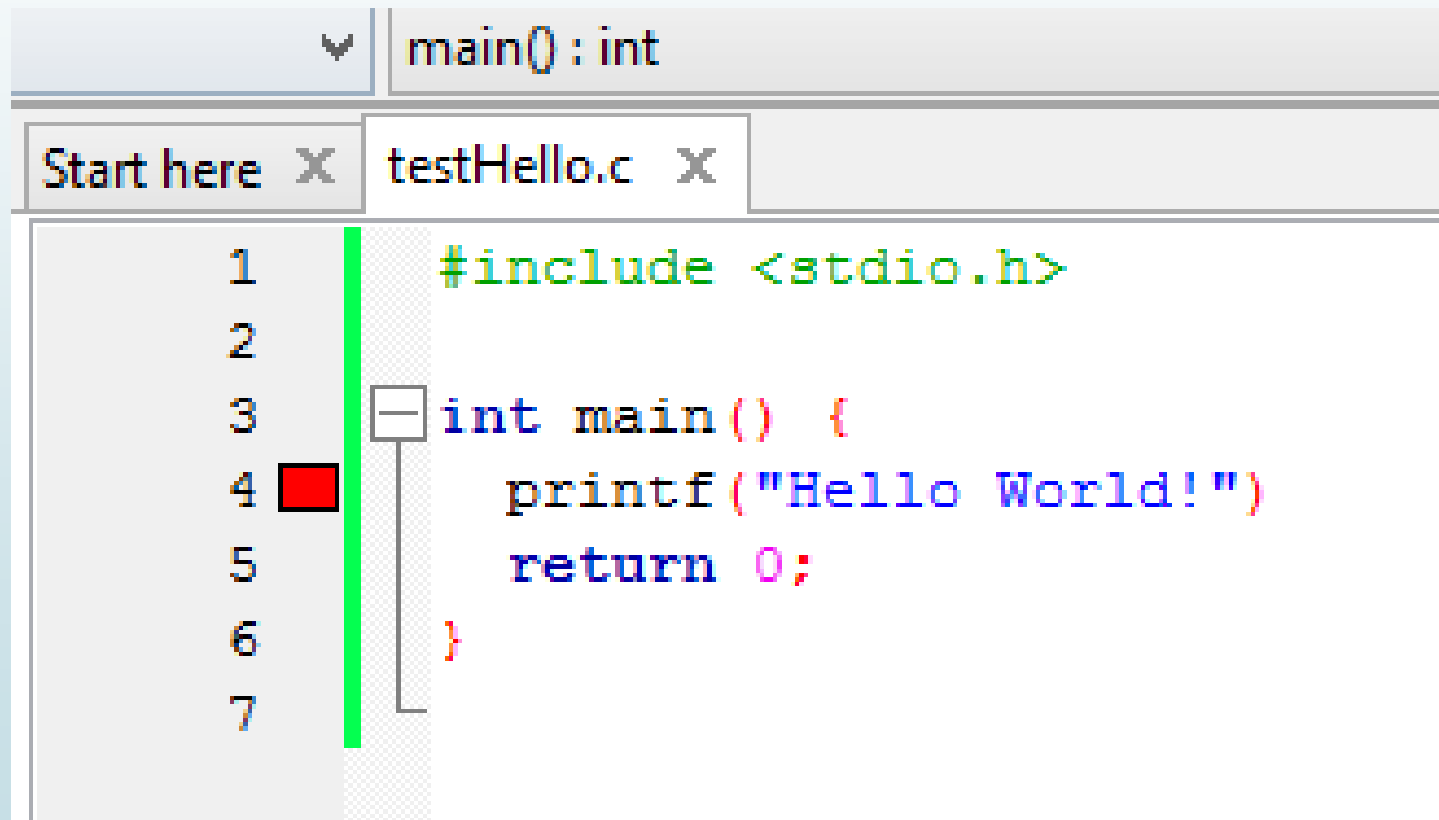
- `printf("Hello World! ");`

;

"

Syntax Error

Example missing semicolon (;)



The screenshot shows a code editor window with a single file named 'testHello.c'. The code is as follows:

```
1  #include <stdio.h>
2
3  int main() {
4      printf("Hello World!")
5      return 0;
6  }
7
```

A red square cursor is positioned at the end of line 4, indicating the location of the syntax error. A red bracket is drawn around the closing parenthesis of the 'main' function on line 6, highlighting the missing semicolon. The editor's status bar at the top shows 'main() : int'.

Example of the error message

```
prog.c: In function 'main':  
prog.c:4:25: error: expected ';' before 'return'  
    4 |     printf("Hello World!")  
      |                               ^  
      |                               ;
```


Runtime Errors



- Errors that are detected when your program is running, but not during compilation.
- When the computer detects an error, it terminates the program and prints an error message.
- Example: attempting to divide by 0.
total = 90/0;

Logic Errors

- Errors that are not detected during compilation or while running but cause the program to produce incorrect results.
- Example: an attempt to calculate the total of groceries that you buy, instead of summing the values, you subtract the values.

EXERCISE

Write this program:

1. Define the class name as **DisplayMessage**.
2. Put single line comment about the class:
This class displays the students' interest
3. Put multiple line comment to display message:
Display a message in the console window
4. The program will display this message:
I like PROGRAMMING in JAVA very much!

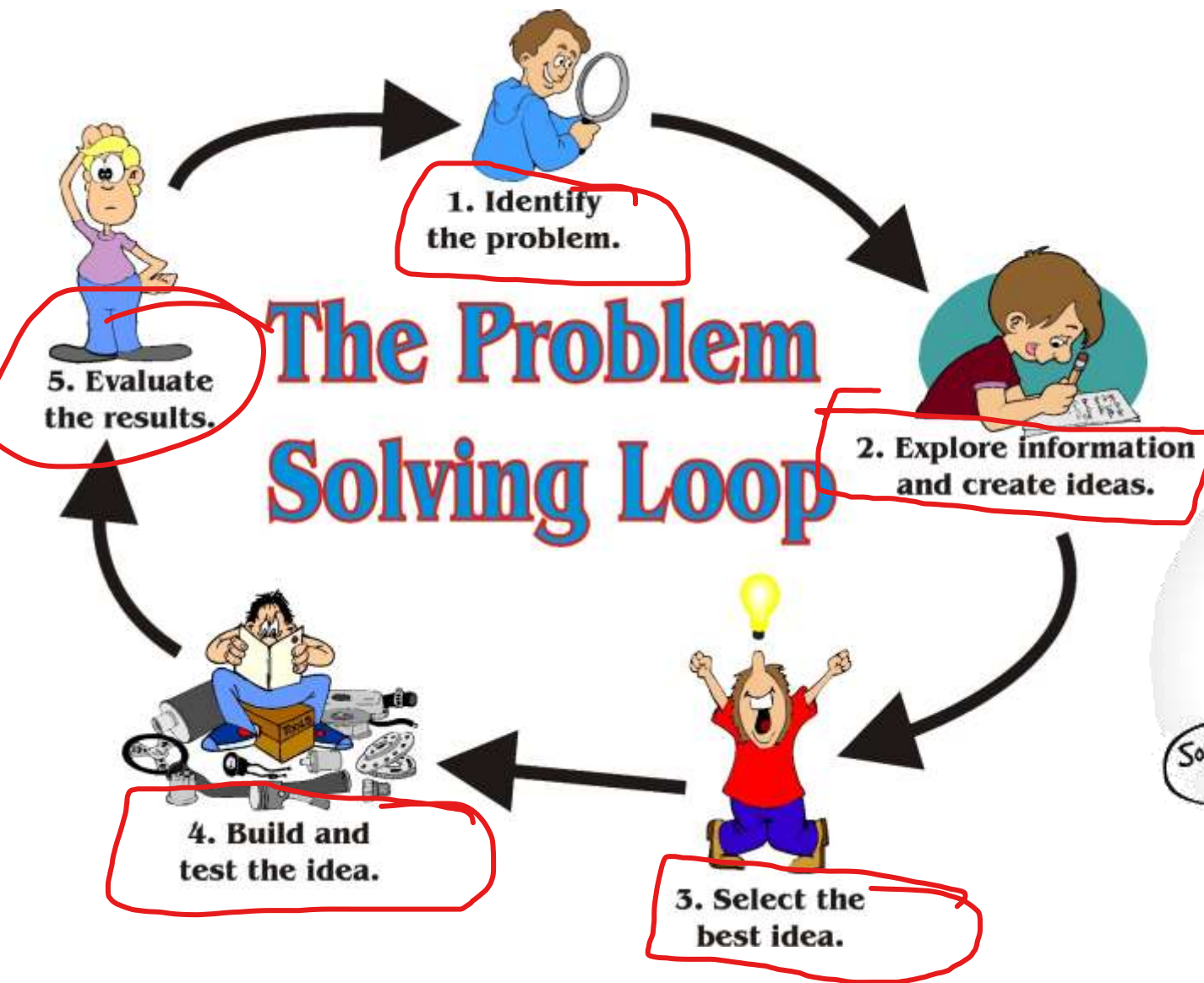
TODAY'S TAKE AWAY



- ✓ Understand what is programming
- ✓ Learn about the architecture of computers
- ③ ✓ Learn about level of programming languages
- ✓ Learn about computing environment and your compiler
- ✓ Learn how to compile and run your first Java program
- ~~①~~ ✓ Recognize 3 types of errors

INTRODUCTION TO PROBLEM SOLVING





Objectives

- 1) To describe nature of problems:
 - Concrete & Abstract
- 2) To learn about the types of solution for a problem
 - Algorithmic & Heuristic Solutions
- 3) To learn the steps involved in building a program
- 4) To learn problem solving through algorithm
 - pseudo code & flow chart



Introduction



The most important skill for a computer scientist is **problem-solving:**

the ability to formulate problems, think creatively about solutions, and express a solution clearly and accurately.



Programming involves

problem solving (design)

translating the solutions into a computer program using a programming language (a.k.a . coding)

Nature of Problem

- A **problem** in computer science is the statement of specifications that requires a solution
- Problem can be **concrete** or **abstract**
- **Concrete**: they have known values to use in making the solution:

i) Compute the area of a circle when its radius is 5.5 cm

$$\begin{aligned}\text{Circle area} &= r * r * 3.142 \\ &= 5.5 * 5.5 * 3.142 \\ &= 95.046\text{cm}^2\end{aligned}$$

ii) Calculate volume of a rectangle when the length is 12cm, width is 8cm and height is 6cm

$$\begin{aligned}\text{Rectangle Volume} &= \text{Length} \times \text{Width} \times \text{Height} \\ &= 12 \times 8 \times 6 \\ &= 576 \text{ cm}^3\end{aligned}$$

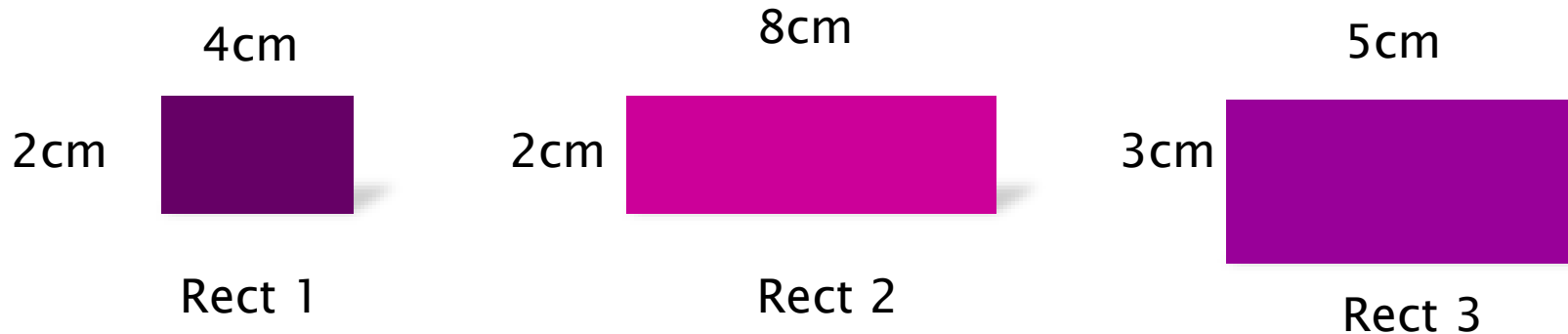
The solution is designed to solve a single specific problem

Cont..

• Abstract: they have no particular values to be used in making the solution:

- i) Compute the area of a circle with a given radius
 - ii) Calculate the monthly payment for a given loan amount with a specific loan duration and interest rate.
- requires an abstract/general solution which can **solve several problems** when concrete information is given.

ANALOGY: TO CALCULATE THE AREA OF 3 DIFFERENT RECTANGLES WITH GIVEN VALUES



Area = $2 * 4$

Area = $2 * 8$

Area = $3 * 5$

3 different Java programs

Concrete Problem

ANALOGY 2: TO CALCULATE THE AREA OF RECTANGLES



1 Java program
for 3 rectangles
calculation

Abstract
Problem

Find general solution:

$$\text{Rect area} = \boxed{\text{Width}} * \boxed{\text{Length}}$$

$$\text{Rect 1} = \boxed{2} * \boxed{4}$$

$$\text{Rect 2} = \boxed{2} * \boxed{8}$$

$$\text{Rect 3} = \boxed{3} * \boxed{5}$$

Concrete
information



TYPES OF PROBLEM SOLUTIONS: **ALGORITHMIC SOLUTIONS**

- Well-defined steps/instructions for carrying out a particular task .
 - E.g. Calculating your bank account balance
 - Calculating the area of a circle
 - Calculating total salary
- The solution will be the same each time the algorithm is followed.
- Most computers use algorithmic types of problems.



TYPES OF PROBLEM SOLUTIONS: **HEURISTIC SOLUTION**

- Solutions that can't be reached by following a direct set of steps.
 - Create different solutions each time program runs
 - Created by using reasoning built upon knowledge and experience /situation.
 - E.g. - Determining which road to take to reach UUM
 - Deciding which group to choose for a subject
- The solution may not be the same each time the algorithm is executed.
- Artificial intelligence deals with heuristic types of problems.

Heuristic

Algorithmic

VS

Heuristic
Solutions

Algorithmic
Heuristic

■ **Algorithmic solution for driving to someone's house:**

Take the Highway to Kedah. Take the Alor Star Utara exit and drive 4 miles. Turn left at the grocery store, and then take the first left. Drive along the way and you will find the house on the left, at Lot 2.

■ **Heuristic solution for getting to someone's house:**



Find the last letter we mailed to you. Drive to the town in the written address. When you get to town, ask someone where our house is. Everyone knows us—someone will be glad to help you. If you can't find anyone, call us and we'll come to get you.

Heuristic

Algorithmic VS Heuristic Solutions



*Algorithmic
Heuristic*



An algorithmic solution gives you the instructions directly.

A heuristic solution tells you how to discover the instructions for yourself, or at least where to look for them.



Algorithmic Solutions

STEPS IN BUILDING A PROGRAM

1

Analyze the problem



2

Develop an algorithm



3

Test an algorithm



4

Code the program



5

Test the program (by programmer and the user)



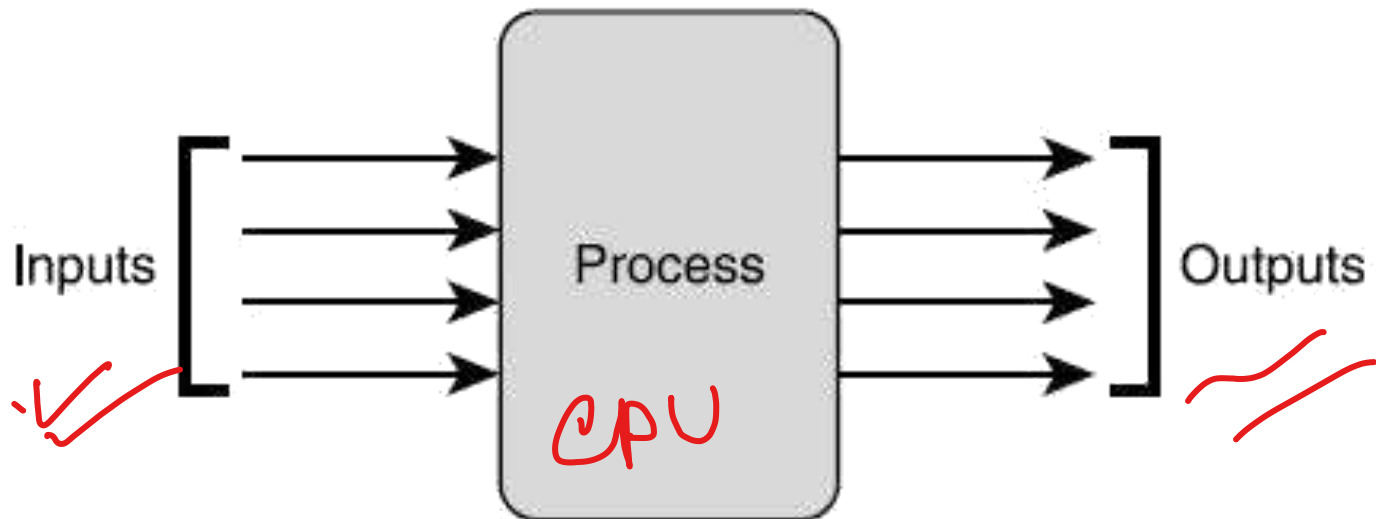
STEPS IN BUILDING A PROGRAM

✓ 1. Analyze the Problem

- Clearly understand the problem
 - what input are to be used
 - the desired output
 - the procedure that will produce the result
 - the constraints to be considered.
-
- The analysis starts by answering these questions:
 - What would be considered the input data?
 - What would be considered the output information?
 - What are the formulas/processes you have to create to solve this solution?
 - Are there any special conditions/constraints?

STEPS IN BUILDING A PROGRAM

Analyze the Problem



EXAMPLE

Write a C program that calculate the perimeter of a rectangle based on the length and width given by user. The program will display the calculated perimeter. The perimeter is calculated by using this formula:

$$\text{Perimeter} = 2 \times (\text{length} + \text{width})$$

- ✓✓ Input? → length, width
- ✓✓ Process? → $\text{Perimeter} = 2 \times (\text{length} + \text{width})$
- ✓✓ Output? → Perimeter

STEPS IN BUILDING A PROGRAM

2. Develop an Algorithm

pseudocode
flowchart

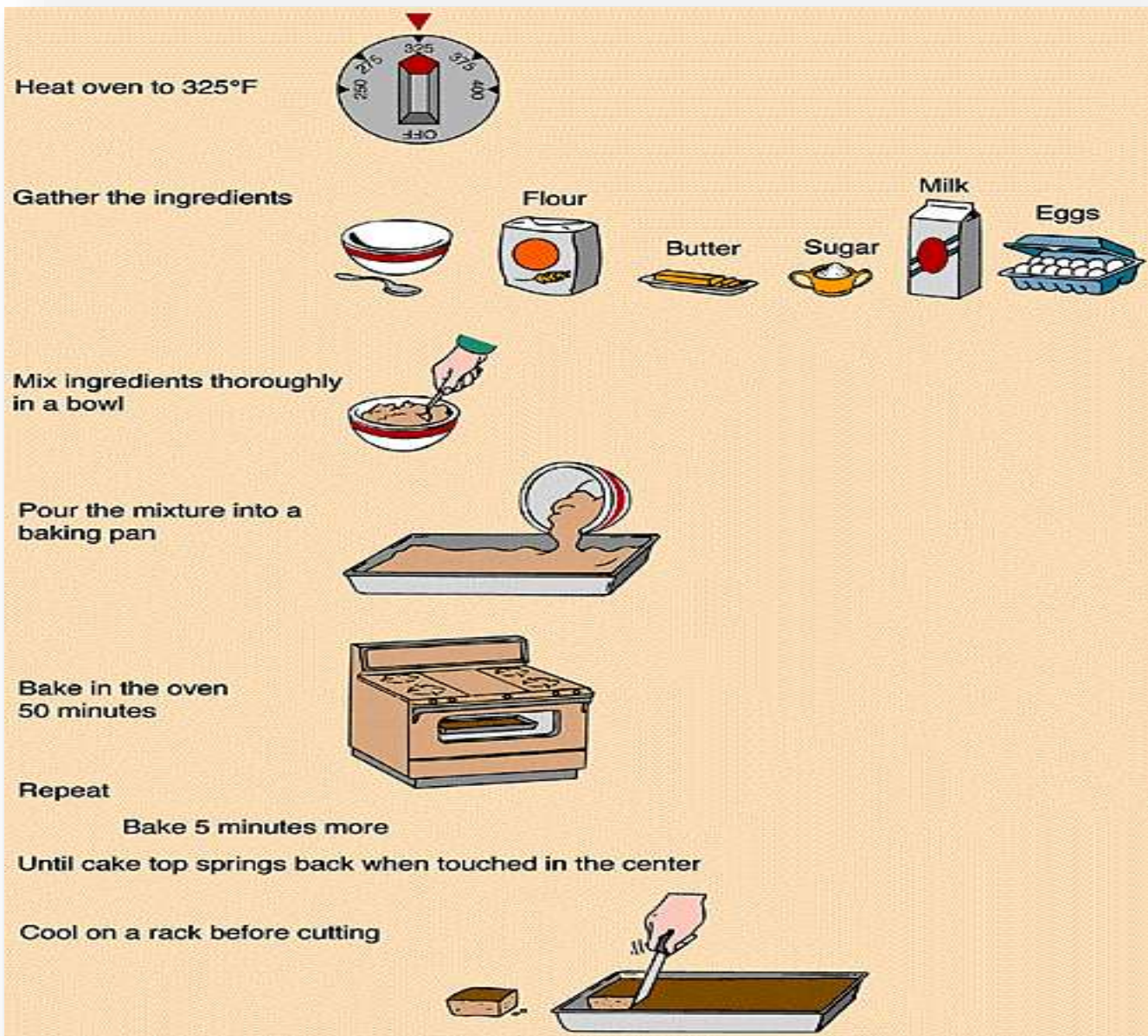
- Write the algorithm in pseudocode or flowchart.
- Need to identify the basic steps clearly in the correct order from what you found in the analysis step.
- Typically, the steps follows similar pattern:
input, process and output.

ALGORITHM

- An algorithm is a clear step-by-step sequence of instructions to solve a problem
- Analogy: recipe (refer next slide).
- In computing, an algorithm is a procedure (a finite set of a well-defined instructions) for accomplishing some task which, given an initial state, will terminate in a defined end-state.

*Algorithmic
(A')*

✓✓ ALGORITHM: BAKING A CAKE



ALGORITHM

■ 2 ways to represent algorithm:

pseudo

~~a)~~ **pseudo code** ✓

using English-like statements to list the steps to be taken in the solution.

~~b)~~ **flowchart** ✓

using diagrams that employ symbols to describe the workflow of steps involved in the solution.

PSEUDOCODE

pseudo

- Pseudocode statements usually perform input, output, and processing.

- The **input** statements are to get data needed to perform computation e.g.:

- Input studentName, studentMark
- Input length, width, height of a box
- Input loan_amount, interest_rate, loan_duration

- The **output** statements are to display information for the user e.g.:

- Output finalGrade
- Output "The volume of the box is " +theVolume
- Output "The loan monthly payment " +monthlyPayment

PSEUDOCODE

■ The **processing** statements are to perform computation e.g.:

i. $\text{priceAverage} = (\text{price1} + \text{price2} + \text{price3}) / 3$

ii. $\text{volume} = \text{length} \times \text{width} \times \text{height}$

iii. $\text{BMI} = \text{weight} / (\text{height} \times \text{height})$

*length x width x
height*

*weight
height²*

More info: https://users.csc.calpoly.edu/~jdalbey/SWE/pdl_std.html

Pseudocode to Calculate the Perimeter of Rectangle

pseudocode

Start

Input length, width

Calculate perimeter = $2 \times (\text{length} + \text{width})$

Output perimeter

End

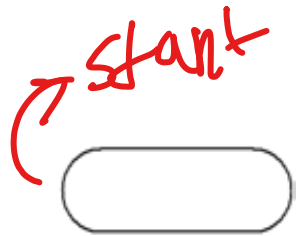
start → take input
→ process
end → output

Flowchart

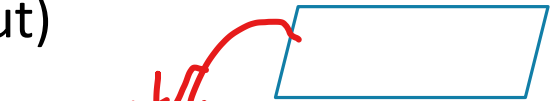
- A graphical way of depicting a problem in terms of its inputs, outputs, and processes.

- The basic elements are as follows:

- ❖ Rounded Rectangle (start/end of a program)



- ❖ Parallelogram (program input and output)



- ❖ Rectangle (processing)



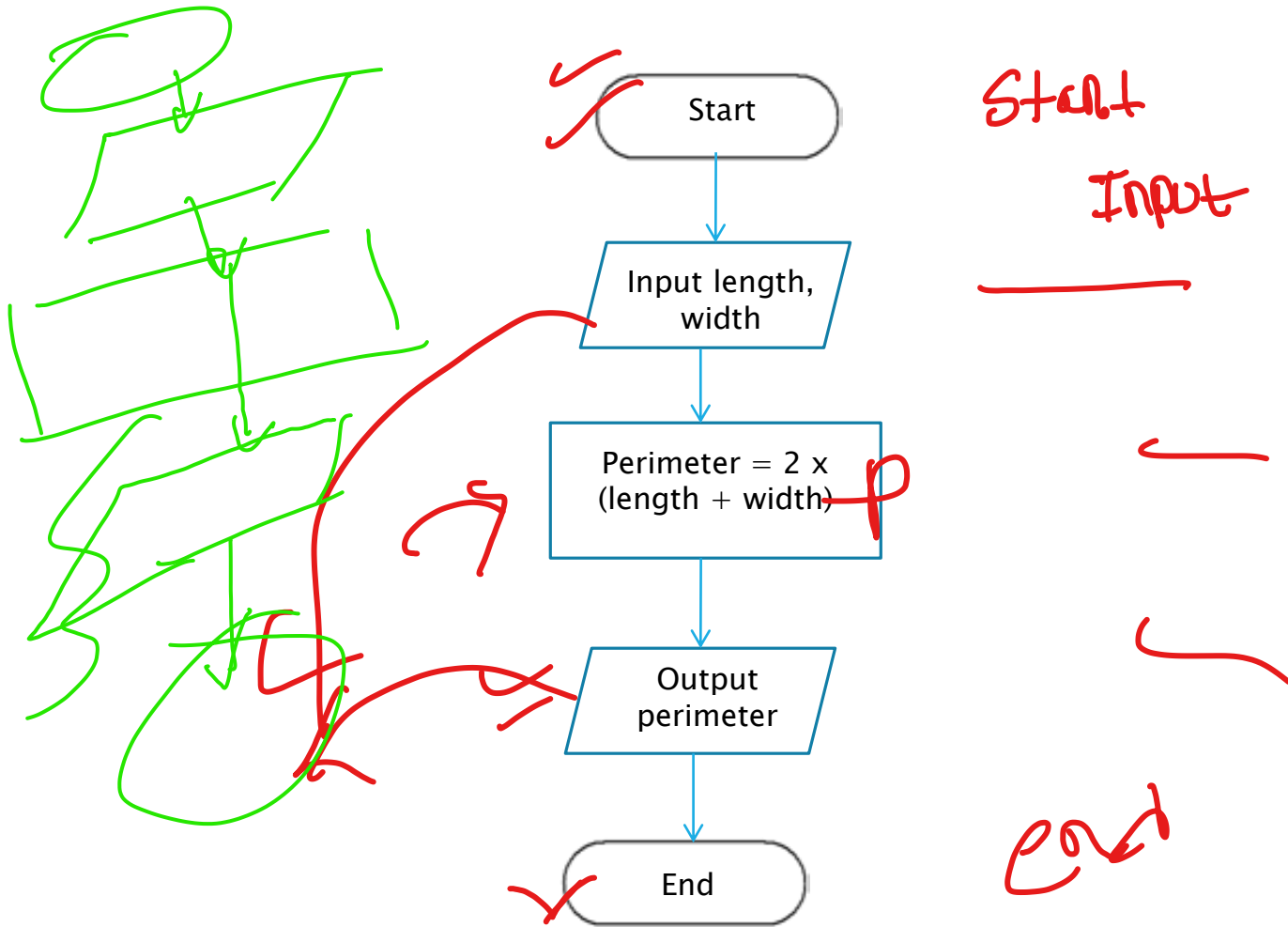
- ❖ Diamond (decision)



- ❖ Arrow (execution flow)



Flowchart to Calculate the Perimeter of a Rectangle



Steps in Building a Program

3. Test an Algorithm:

- Pretend that you are the computer and follow the steps of your algorithm explicitly while keeping track of how the variables are changing.
- Hand trace the values of the variables in every steps of your algorithm to verify its correctness.

TEST ALGORITHM TO CALCULATE THE PERIMETER OF A RECTANGLE

START

INPUT length, width $\rightarrow$ (10,20)

CALCULATE perimeter = $2 \times (\text{length} + \text{width}) \rightarrow 2 \times (10+20)$

OUTPUT perimeter $\rightarrow$ 60

END



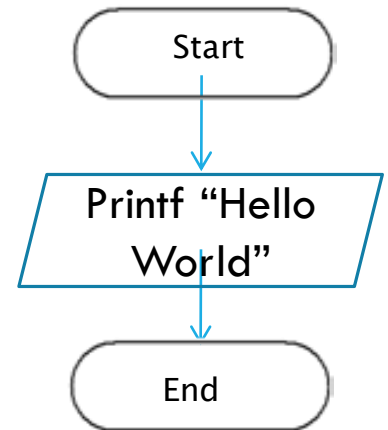
STEPS IN BUILDING A PROGRAM

4. Code the program:

- Develop the program by coding the algorithm in a programming language, such as Java.
- Need to have knowledge in the programming language in terms of the language syntax.
- Can use IDEs such as Eclipse, NetBeans, JGRASP, etc to facilitate the coding tasks.

HELLO WORLD PSEUDOCODE & JAVA CODE

✓✓
START
Display "Hello World!"
END



✓✓

```
public class Hello
{
    public static void main(String[] args)
    {
        // display a greeting in the console window
        System.out.println("Hello World!");
    }
}
```

Example of the Code



```
#include <stdio.h>
```

```
int main() {  
    printf("Hello World!");  
    return 0;  
}
```

C language



```
public class Hello  
{  
    public static void main(String[] args)  
    {  
        System.out.println("Hello World!");  
    }  
}
```

Java language

ALGORITHM & JAVA CODE TO CALCULATE THE PERIMETER OF A RECTANGLE

START
INPUT length, width
CALCULATE perimeter = 2 x (length + width)
OUTPUT perimeter
END

```
#include <stdio.h>
```

```
int main() {  
    int length, width, perimeter;
```

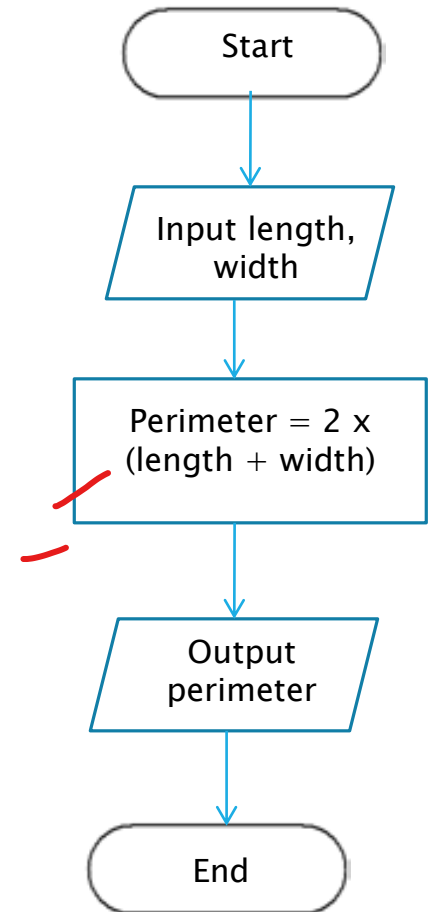
```
    printf("Insert the length: ");  
    scanf("%d", &length);
```

```
    printf("Insert the width: ");  
    scanf("%d", &width);
```

```
    perimeter = 2 * (length + width);
```

```
    printf("The perimeter of the rectangle is: %d\n", perimeter);  
    return 0;
```

```
}
```



STEPS IN BUILDING A PROGRAM

5. Test/Debug Program:

- Test to verify that the program meets the requirements by creating test cases. (Compare the hand-traced value with the output from the code)
- Test can avoid bugs such as logic errors.
- Program also need to be maintained by modifying it if the problem domain or requirements changes or problem arises.



PROBLEM 1

Write pseudo code and flow chart for a program which calculates the ticket price for bowling players. The program should read the number of players. The ticket price per player is RM5. The total price should be displayed.



ANALYZE...

Input? → number of players

Process? → price = number of players * 5

Output? → price

PSEUDO CODE & FLOW CHART

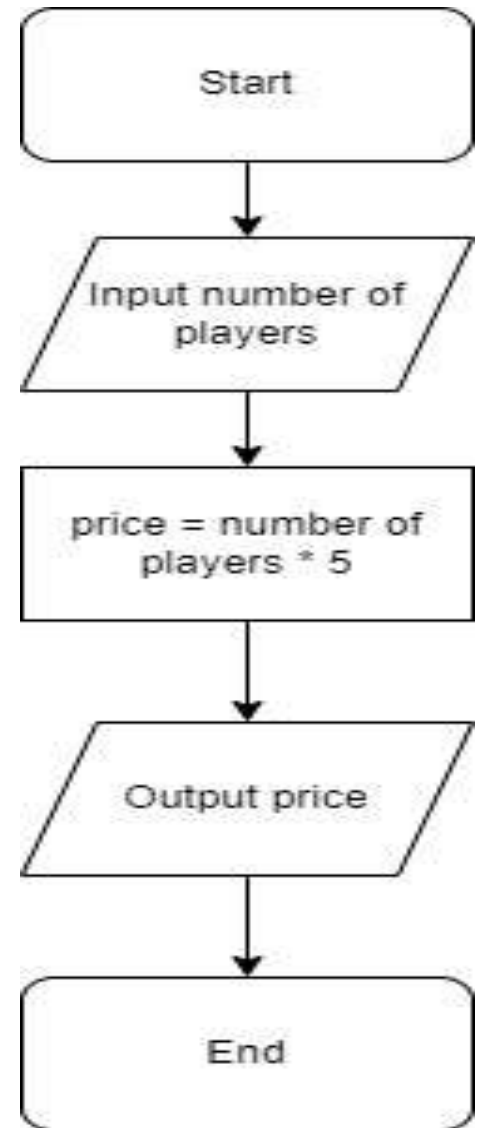
START

INPUT number of players

CALCULATE price = number of players * 5

OUTPUT price

END





PROBLEM 2

Write pseudo code and flow chart for a program which calculates the ticket price for bowling players. The program should read the number of players. The ticket price per player is RM5. The total price should be displayed. If the ticket price is more than RM50, a 10% discount should be given.

ANALYZE...

Input? → number of players

Process? → $\text{price} = \text{number of players} * 5$

Output? → price

Condition? → $\text{price} > 50$, 10% discount

PSEUDO CODE

START

INPUT number of players

CALCULATE price = number of players * 5

IF price > 50 THEN

total = price - (10/100* price)

ELSE

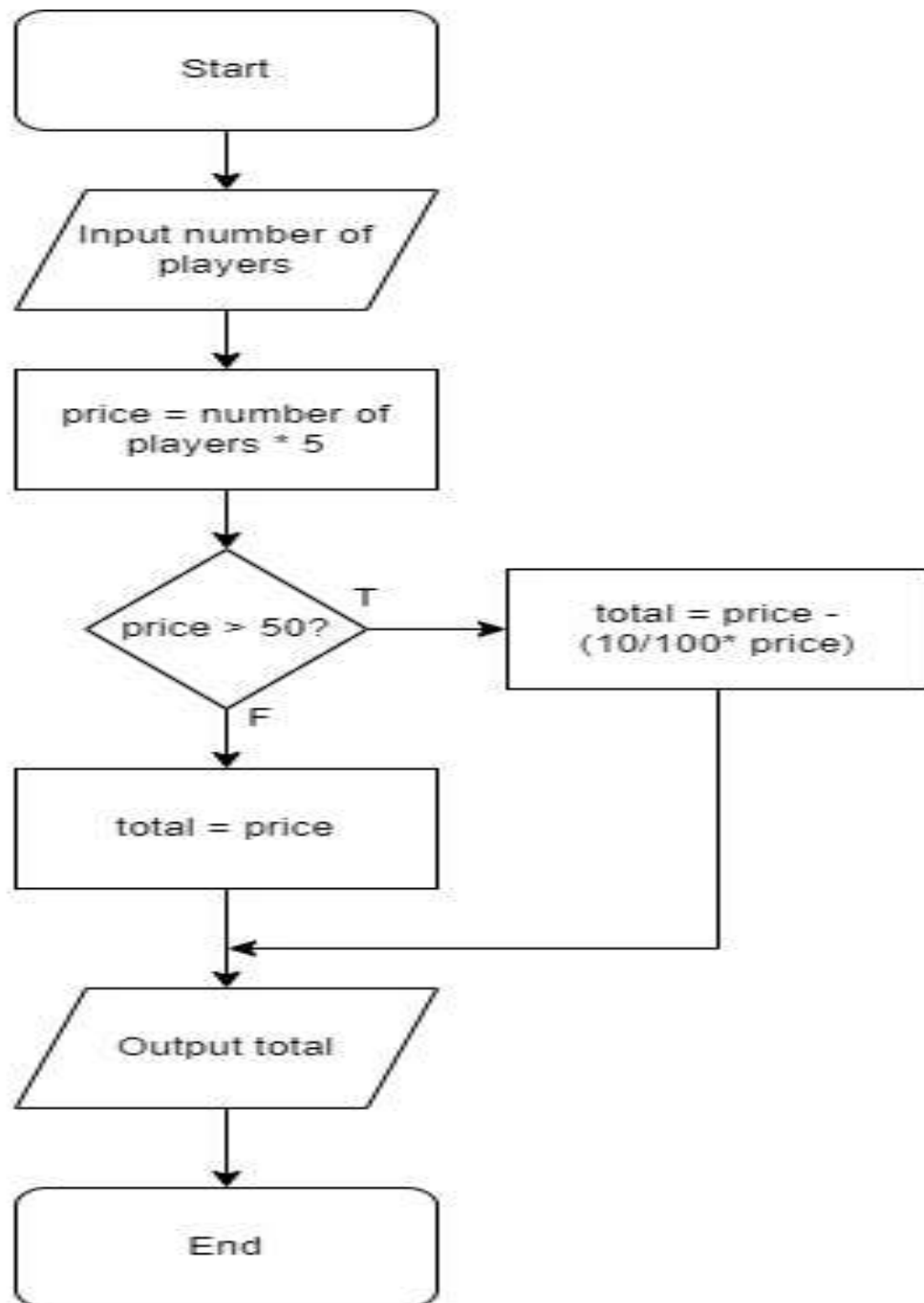
total = price

OUTPUT total

ENDIF

END

FLOW CHART



$$\frac{10}{100} \times 50$$

$$\frac{50}{100} \times 50$$



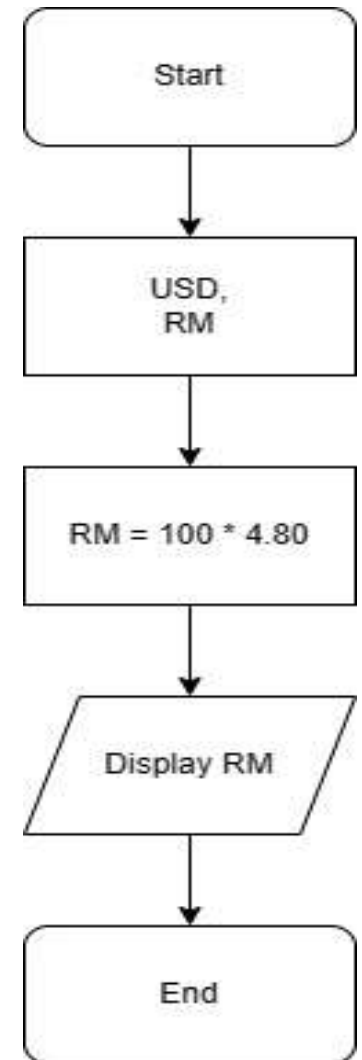


PROBLEM 3

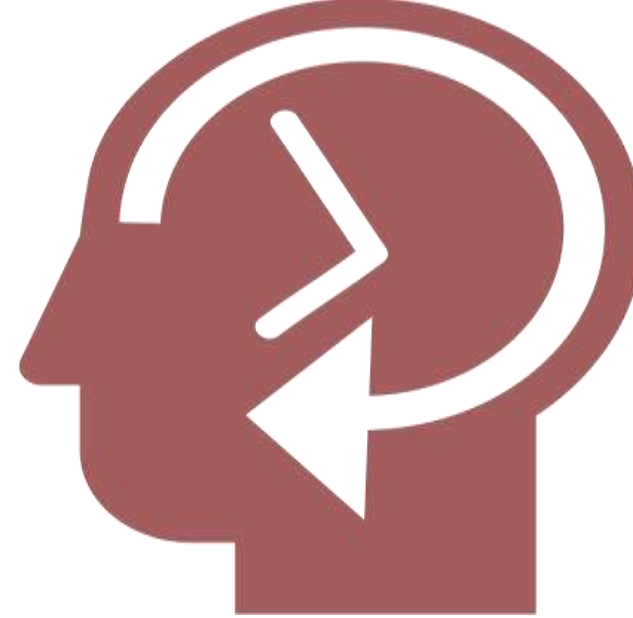
Convert \$100 US Dollar (USD) to Malaysian Ringgit (RM). Assume that USD1 is equivalent to RM4.80. Display RM value calculated by your program.

PSEUDO CODE & FLOW CHART

```
START  
  DECLARE USD, RM  
  CALCULATE  $RM = 100 * 4.80$   
  OUTPUT RM  
END
```



TODAY'S TAKE AWAY



- ✓ Two types of problems nature
- ✓ Types of solution for a problem
- ✓ Steps in building a program



EXERCISE 1

Write pseudo code and flow chart for a program which calculates the carry marks for a subject.

The formula used to calculate the carry marks is:

$$\text{carryMarks} = \text{midSem} + \text{assignment} + \text{quiz}$$



EXERCISE 2

Read the length, width and height of a box, calculate and display its volume. The volume is calculated by multiplying the length, width and height .



EXERCISE 3

Produce algorithm by using pseudo code and flow chart, for a program that calculates and displays a circle area based on its radius. The formula to calculate area is πr^2 where r = radius and $\pi = 3.142$.



EXERCISE 4

Ask user to input an amount in US Dollar (USD), read it and then convert to Malaysian Ringgit (RM). Assume that USD1 is equivalent to RM4.20. Display RM value calculated by your program.



EXERCISE 5

Read **three** test scores for a student. Find the average score achieved for that student.



EXERCISE 6

You are asked to develop a simple cashier machine to calculate total payment for grocery bought by customers. You need to read the name of item, its price and quantity. Then the item name and total price for the item are displayed.



EXERCISE 7

You would like to develop a simple ATM Machine for withdrawing money. The user of this ATM Machine should provide the total balance in the account before withdrawal and the total withdrawal amount. At then end, the ATM machine will display the total account before withdrawal, withdrawal amount and balance after withdrawal.



EXERCISE 8

You are asked to develop a system for Legolands Theme Park. This system is supposed to calculate the total price that need to be paid by visitors. The ticket price for adult is RM50 while for children, the ticket price is RM30.



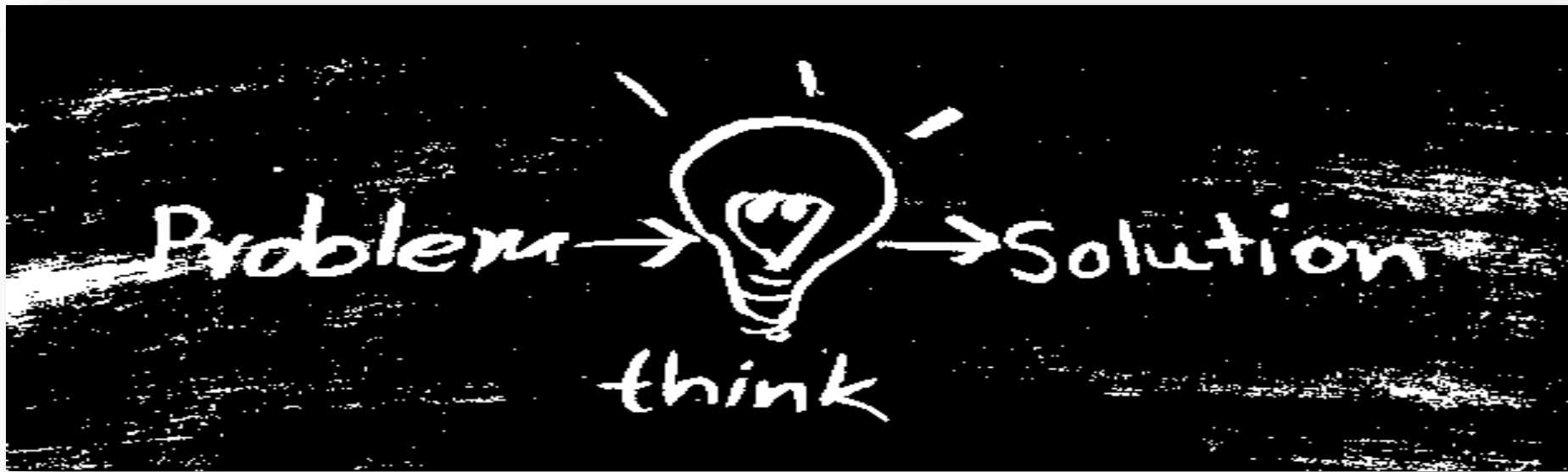
EXERCISE 9

You are asked to develop a car parking system for RSF Mall. You need to calculate the payment for parking a car in the car park based on hours parked. Each hour is charged RM1.50. If the hours parked is 24, then RM30 should be charged. Calculate and display the total payment that need to be paid when a user parks the car.



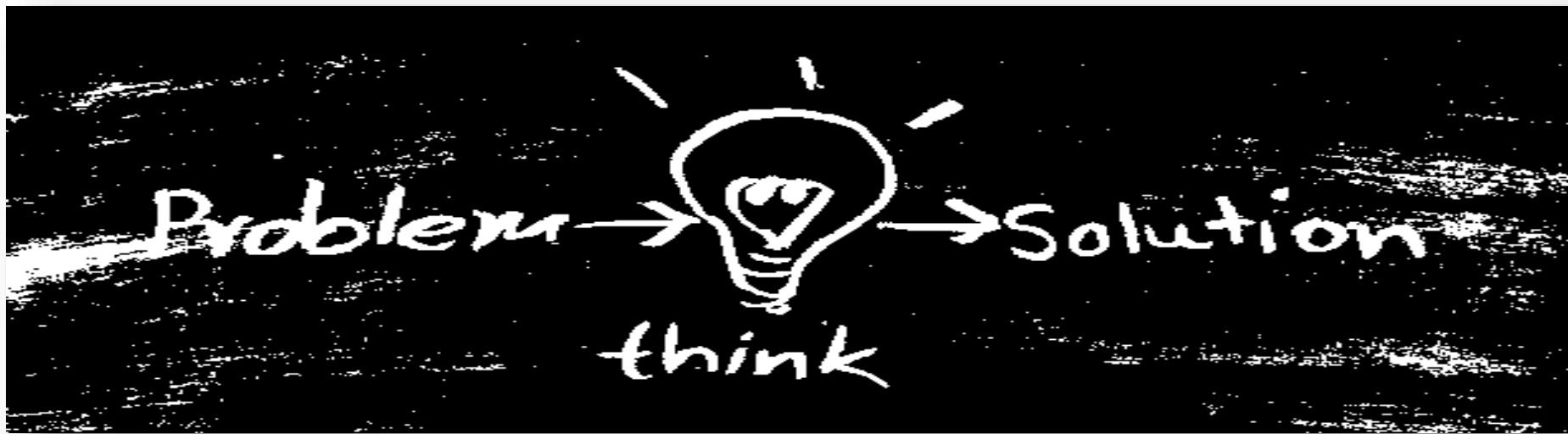
EXERCISE 10

Read the actual price of a Bonia handbag, its discount percentage and membership status. If the customer has a membership, another RM10 is deducted. Calculate and print the price of handbag that need to be paid.



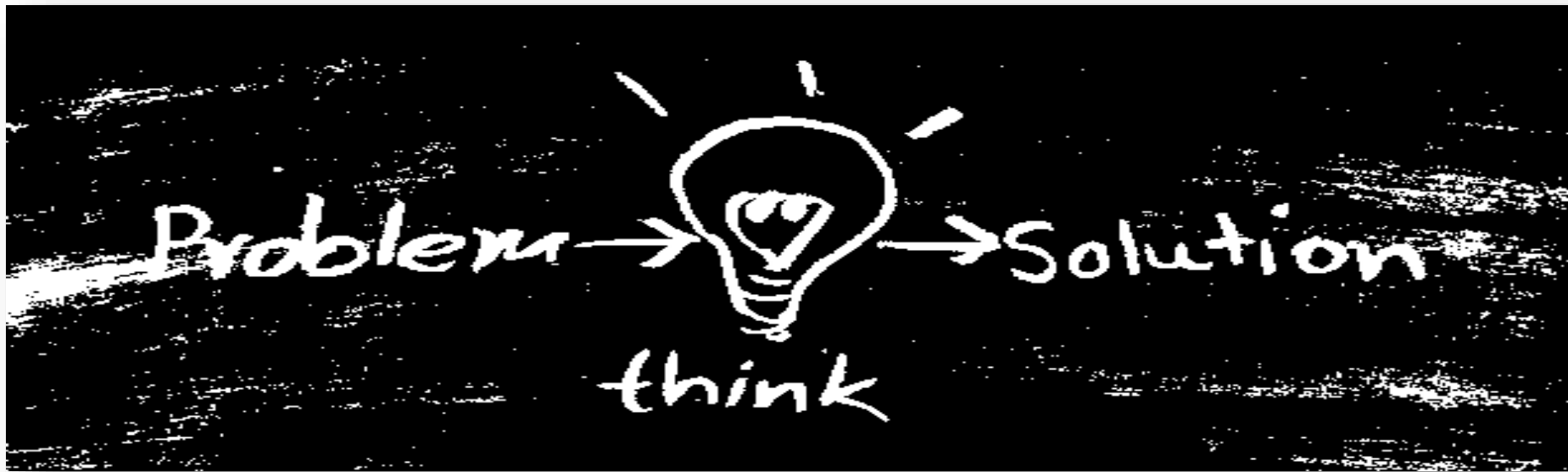
EXERCISE 11

You are asked to develop a system for UUM Go-kart. This system is supposed to calculate the total price that need to be paid by 5 UUM students and 2 outsiders. The ticket price for UUM students is RM5 while for outsider, the ticket price is RM10.



EXERCISE 12

The actual price for ASUS laptop is RM3450. If the customer is a university student, 15% is deducted. Calculate and print the price of the laptop that need to be paid.



EXERCISE 13

A customer buys 10 books and 15 sets of coloring pencils. Each book costs RM30 while the coloring pencil is RM7 per set. You are asked to develop a system which can calculate and display the total price for these items.

Fundamental of C and Basic Input Output CCC 1123

*Asmaul
Hosen*

Baharudin Osman

A decorative graphic on the left side of the slide. It consists of a blue parallelogram and a light green parallelogram, both tilted at an angle. The blue shape is in the foreground, and the green shape is partially behind it. They are set against a dark blue background with faint, lighter blue diagonal stripes.

Basic Element of Computer Programs



Elements of c language

Preprocessor directives

Keywords

Identifier

Variables

Statements

Comments

Operator

Functions

Preprocessor directives

- A pre-processor directive begins with a special character # (hash/pound). When you compile the code, then before compilation, directives tell the preprocessor to perform specific actions.
- These actions can be like replacing tokens in the text (using the macro), insert the contents of other files into the source file (Using file inclusion like `#include <stdio.h>`), or perform conditional Compilation (using `#if`, `#else`, `#endif` ..etc).

Standard C keywords

Keywords are predefined, reserved words that have special meanings to the compiler. Keywords are part of the syntax and they cannot be used as an identifier.

As C is a case sensitive language, all keywords must be written in lowercase. Here is a list of all keywords allowed in ANSI C.

<u>auto</u>	double ✓	int ✓	<u>struct</u>
<u>break</u>	else	long ✓	<u>switch</u>
case	enum	<u>register</u>	<u>typedef</u>
char	extern	return	<u>union</u>
continue	for	<u>signed</u>	void
do	if	<u>static</u>	while
default	goto	sizeof	<u>volatile</u>
const	float	short	<u>unsigned</u>

Identifiers

- Identifier refers to name given to entities such as **variables**, **functions**, **structures** etc.
- Identifiers must be unique. They are created to give a unique name to an entity to identify it during the execution of the program. For example:

int money;

double accountBalance;

- Here, **money** and **accountBalance** are identifiers.
- Also remember, identifier names must be different from keywords. You cannot use **int** as an identifier because **int** is a keyword.

Rules for naming identifiers

1. A valid identifier can have **letters (both uppercase and lowercase letters), digits and underscores**.
2. The **first letter** of an identifier should be either a **letter or an underscore**.
3. You cannot use **keywords** like `int`, `while` etc. as identifiers.
4. There is no rule on how long an identifier can be. However, you may run into problems in some compilers if the identifier is **longer than 31 characters**.
5. There is **no space** between the identifier names
6. A identifier name cannot start with a digit but you use it between the identifier
7. You cannot use **special character** between identifier, but you can use underscore anywhere in the identifier
8. A **variable name can not same as function name** within the same scope

You can choose any name as an identifier if you follow the above rule, however, give meaningful names to identifiers that make sense.



Variables and Constants

Variable

A variable is a container that defines a memory location to hold data and we can use these data whenever required in the program. In other words, we can say that variable is a name (or identifier) which indicate the physical address in the memory, where data will be stored in form of the bits of string. The data / value of a variable can be changed at different times of executions and it may be chosen by the programmer in a meaningful way.

Syntax of variable: **DataType** VariableName;

e.g. : **int** mathMarks = **99**;

Here, mathMarks is a variable of **int** type and the variable is assigned with integer value **99**.

The value of a variable can be changed, eg.

mathMarks = **95**;



Example

```
#include <stdio.h>

int main() {
    int myNum = 15;
    printf(myNum); // Nothing happens
    return 0;
}
```

Variable in C must be a specified **data type**, and you must use a **format specifier** inside the **printf()** function to display it:

Data Types in C

Each variable in C has an associated data type. It specifies the type of data that the variable can store like integer, character, floating, double, etc. Each data type requires different amounts of memory and has some specific operations which can be performed over it. Data types in C can be classified as follows:

Types	Description	Data Types
Primitive Data Types	Primitive data types are the most basic data types that are used for representing simple values such as integers, float, characters, etc.	int, char, float, double, void
<u>Derived Types</u>	The data types that are derived from the primitive or built-in datatypes are referred to as Derived Data Types.	array, pointers, function
<u>User Defined Data Types</u>	The user-defined data types are defined by the user himself.	structure, union, enum

primitive
Derived
User defined
data type

Cont..

The data type specifies the **size** and **type** of information the variable will store.

char - 1
int - 2 or 4
float - 4
double - 8

Data Type	Size	Description	Example
int	2 or 4 bytes	Stores whole numbers, without decimals	1
float	4 bytes	Stores fractional numbers, containing one or more decimals. Sufficient for storing 6-7 decimal digits	1.99
double	8 bytes	Stores fractional numbers, containing one or more decimals. Sufficient for storing 15 decimal digits	1.99
char	1 byte	Stores a single character/letter/number, or ASCII values	'A'

Basic Format Specifiers

Format Specifier	Data Type
<code>%d</code> or <code>%i</code>	int → <code>%d</code> / <code>%i</code>
<code>%f</code> or <code>%F</code>	float <code>%f</code> / <code>%F</code>
<code>%lf</code>	double <code>%lf</code>
<code>%c</code>	char <code>%c</code>
<code>%s</code>	Used for strings (text) , which you will learn more about in a later chapter <code>%s</code>

Size of `%s` → lu

`%d`
✓
✓
✓
✓
✓

Example

Example

```
// Create variables
int myNum = 5; // Integer (whole number)
float myFloatNum = 5.99; // Floating point number
char myLetter = 'D'; // Character

// Print variables
printf("%d\n", myNum);
printf("%f\n", myFloatNum);
printf("%c\n", myLetter);
```

Range and Format Specifiers of data types

Data Type	Memory (bytes)	Range	Format Specifier
short int	2	-32,768 to 32,767	%hd
unsigned short int	2	0 to 65,535	%hu
unsigned int	4	0 to 4,294,967,295	%u
int	4	-2,147,483,648 to 2,147,483,647	%d
long int	4	-2,147,483,648 to 2,147,483,647	%ld
unsigned long int	4	0 to 4,294,967,295	%lu
long long int	8	$-(2^{63})$ to $(2^{63})-1$	%lld
unsigned long long int	8	0 to 18,446,744,073,709,551,615	%llu
signed char	1	-128 to 127	%c
unsigned char	1	0 to 255	%c
float	4	3.4E-38 to 3.4E+38	%f
double	8	1.7E-308 to 1.7E+308	%lf
long double	10	3.4E-4932 to 1.1E+4932	%Lf

Numeric Types

- Use **int** when you need to store a whole number without decimals, like 35 or 1000,
- Use **float** or **double** when you need a floating point number (with decimals), like 9.99 or 3.14515.

int

1000

```
int myNum = 1000;  
printf("%d", myNum);
```

float

5.750000

```
float myNum = 5.75;  
printf("%f", myNum);
```

double

19.990000

```
double myNum = 19.99;  
printf("%lf", myNum);
```



float vs double

- The **precision** of a floating point value indicates how many digits the value can have after the decimal point.
- The precision of **float** is **six** or **seven** decimal digits, while **double** variables have a precision of about **15** digits.

Scientific Numbers

- A floating point number can also be a scientific number with an "e" to indicate the power of 10:

Example

```
float f1 = 35e3;  
double d1 = 12E4;
```

```
printf("%f\n", f1);  
printf("%.1f", d1);
```

35000.000000

120000.000000

Set Decimal Precision

- You have noticed that a floating point number will show many digits after the decimal point.
- If you want to remove the extra zeros (set decimal precision), you can use a dot (.) followed by a number that specifies how many digits that should be shown after the decimal point:

Example

```
float myFloatNum = 3.5;
```

```
printf("%f\n", myFloatNum); // Default will show 6 digits after the decimal point
printf("%.1f\n", myFloatNum); // Only show 1 digit
printf("%.2f\n", myFloatNum); // Only show 2 digits
printf("%.4f", myFloatNum); // Only show 4 digits
```

%.2f

3.500000

3.5

3.50

3.5000

Example

Example

```
// Create variables of different data types
int items = 50;
float cost_per_item = 9.99;
float total_cost = items * cost_per_item;
char currency = '$';

// Print variables
printf("Number of items: %d\n", items);
printf("Cost per item: %.2f %c\n", cost_per_item, currency);
printf("Total cost = %.2f %c\n", total_cost, currency);
```

```
Number of items: 50
Cost per item: 9.99 $
Total cost = 499.50 $
```

✓ Type Conversion

- Sometimes, you have to convert the value of one data type to another type. This is known as type conversion.
- For example, if you try to divide two integers, 5 by 2, you would expect the result to be 2.5.

Example

```
int x = 5;  
int y = 2;  
int sum = 5 / 2;  
  
printf("%d", sum); // Outputs 2
```

Output should
be 2.5.
Why ????

conversion is done manually by placing the type in parentheses () in front of the value.

Cont..

- There are two types of conversion in C:
 - **Implicit** Conversion (**automatically**)
 - Implicit conversion is done automatically by the compiler when you assign a value of one type to another.
 - example, if you assign an **int** value to a **float** type:
 - **Explicit** Conversion (**manually**)
 - Conversion is done manually by placing the type in parentheses () in front of the value.

```
// Automatic conversion: int to float
float myFloat = 9;

printf("%f", myFloat); // 9.000000
```

```
// Automatic conversion: float to int
int myInt = 9.99;

printf("%d", myInt); // 9
```

```
// Manual conversion: int to float
float sum = (float) 5 / 2;

printf("%f", sum); // 2.500000
```

```
int num1 = 5;
int num2 = 2;
float sum = (float) num1 / num2;

printf("%f", sum); // 2.500000
```

Char Type

The **char** data type is used to store a **single** character.

Example

```
char myGrade = 'A';  
printf("%c", myGrade);
```

Example

```
char a = 65, b = 66, c = 67;  
printf("%c", a);  
printf("%c", b);  
printf("%c", c);
```

A
B
C

Example

```
char myText = 'Hello';  
printf("%c", myText);
```

If you try to store more than a single character, it will only print the last character:

String Type

- To store multiple characters (or whole words), use strings

Example

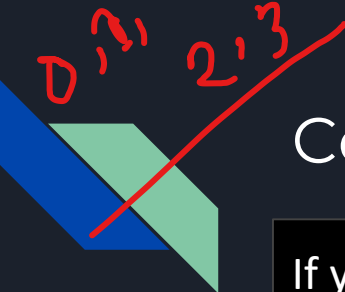
```
char myText[] = "Hello";  
printf("%s", myText);
```

```
char greetings[] = "Hello World!";  
printf("%c", greetings[0]);
```

Modify Strings

To change the value of a specific character in a string, refer to the index number, and use **single quotes**

```
char greetings[] = "Hello World!";  
greetings[0] = 'J';  
printf("%s", greetings);  
// Outputs Jello World! instead of Hello World!
```



Constants

If you want to define a variable whose value cannot be changed, you can use the const keyword. For example,

```
const double PI = 3.14;
```



Notice, we have added keyword const.

Here, **PI** is a symbolic constant; its value cannot be changed.

```
const double PI = 3.14;
```

```
PI = 2.9; //Error
```

↳ const
cannot be
changed

Cont..

- When you declare a constant variable, **it must be assigned with a value**, otherwise you will get an error.

Example

Like this:

```
const int minutesPerHour = 60;
```



This however, **will not work**:

```
const int minutesPerHour;  
minutesPerHour = 60; // error
```



Arithmetic Operators

Operator	Name	Description	Example
+	Addition	Adds together two values	$x + y$
-	Subtraction	Subtracts one value from another	$x - y$
*	Multiplication	Multiplies two values	$x * y$
/	Division	Divides one value by another	x / y
%	Modulus	Returns the division remainder	$x \% y$
++	Increment	Increases the value of a variable by 1	$++x$
--	Decrement	Decreases the value of a variable by 1	$--x$

Relational Operators

Operator	Name	Example	Description
==	Equal to	x == y	Returns 1 if the values are equal
!=	Not equal	x != y	Returns 1 if the values are not equal
>	Greater than	x > y	Returns 1 if the first value is greater than the second value
<	Less than	x < y	Returns 1 if the first value is less than the second value
>=	Greater than or equal to	x >= y	Returns 1 if the first value is greater than, or equal to, the second value
<=	Less than or equal to	x <= y	Returns 1 if the first value is less than, or equal to, the second value

✓ Logical Operators

Operator	Name	Example	Description
&&	AND	<code>x < 5 && x < 10</code>	Returns 1 if both statements are true
	OR	<code>x < 5 x < 4</code>	Returns 1 if one of the statements is true
!	NOT	<code>!(x < 5 && x < 10)</code>	Reverse the result, returns 0 if the result is 1

User Input

To get user input, you can use the `scanf()` function.

```
// Create an integer variable that will store the number we get from the user  
int myNum;
```

```
// Ask the user to type a number  
printf("Type a number: \n");
```

```
// Get and save the number the user types  
scanf("%d", &myNum);
```

```
// Output the number the user typed  
printf("Your number is: %d", myNum);
```

scanf()
fgets(name, sizeof, stdin)

Multiple User Input

```
// Create an int and a char variable
int myNum;
char myChar;

// Ask the user to type a number AND a character
printf("Type a number AND a character and press enter: \n");

// Get and save the number AND character the user types
scanf("%d %c", &myNum, &myChar);

// Print the number
printf("Your number is: %d\n", myNum);

// Print the character
printf("Your character is: %c\n", myChar);
```

The user
input is "5b"



```
Type a number AND a character and press enter: 5b
Your number is: 5
Your character is: b
```

String Input

When working with strings in `scanf()`, you **don't have** to use the **reference operator (&)**.

```
// Create a string
char firstName[30];

// Ask the user to input some text
printf("Enter your first name: \n");

// Get and save the text
scanf("%s", firstName);

// Output the text
printf("Hello %s", firstName);
```

The user
input is
"Baharudin"

```
Enter your first name and press enter:
Baharudin
Hello Baharudin
```

Cont..

scanf() function has some limitations: it considers space (whitespace, tabs, etc) as a terminating character, which means that it only display a single word (even if you type many words).

```
Enter your first name and press enter:
```

```
Baharudin Osman
```

```
Hello Baharudin
```

The user input is
"Baharudin Osman"

From the example above, you would expect the program to print "Baharudin Osman", but it only prints "Baharudin".

That's why, when working with strings, we often use the `fgets()` function to **read a line of text**. Note that you must include the following arguments: the name of the string variable, `sizeof(string_name)`, and `stdin`:

Example

`fgets(stringName, sizeof(stringName), stdin);`

```
char fullName[30];  
  
printf("Type your full name: \n");  
fgets(fullName, sizeof(fullName), stdin);  
  
printf("Hello %s", fullName);  
  
// Type your full name: John Doe  
// Hello John Doe
```

Use the `scanf()` function to get a **single word** as input, and use `fgets()` for **multiple words**.

Escape Sequences Literals

Sometimes, it is necessary to use characters that cannot be typed or has special meaning in C programming. For example: newline(enter), tab, question mark etc.

In order to use these characters, escape sequences are used.

Escape Sequences	Character
<code>\b</code>	Backspace
<code>\f</code>	Form feed
<code>\n</code>	Newline
<code>\r</code>	Return
<code>\t</code>	Horizontal tab
<code>\v</code>	Vertical tab

Escape Sequences	Character
<code>\\</code>	Backslash
<code>\'</code>	Single quotation mark
<code>\"</code>	Double quotation mark
<code>\?</code>	Question mark
<code>\0</code>	Null character

~~Important~~

Arithmetic Operator

Operator	Name	Description	Example
+	Addition	Adds together two values	$x + y$
-	Subtraction	Subtracts one value from another	$x - y$
*	Multiplication	Multiplies two values	$x * y$
/	Division	Divides one value by another	x / y
%	Modulus	Returns the division remainder	$x \% y$
++	Increment	Increases the value of a variable by 1	$++x$
--	Decrement	Decreases the value of a variable by 1	$--x$

Example

```
#include <stdio.h>

int main() {
    int x = 5;
    int y = 2;
    printf("%d", x % y);
    return 0;
}
```



```
#include <stdio.h>

int main() {
    int x = 5;
    printf("%d", ++x);
    return 0;
}
```

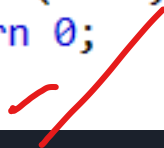


```
#include <stdio.h>

int main() {
    int x = 12;
    int y = 3;
    printf("%d", x / y);
    return 0;
}
```

```
#include <stdio.h>

int main() {
    int x = 5;
    printf("%d", --x);
    return 0;
}
```



Assignment Operator

Operator	Example	Same As
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3

```
#include <stdio.h>
```

```
int main() {  
    int x = 5;  
    x += 3;  
    printf("%d", x);  
    return 0;  
}
```

```
#include <stdio.h>
```

```
int main() {  
    int x = 5;  
    x %= 3;  
    printf("%d", x);  
    return 0;  
}
```



Comments

- Good comments increase the readability of the code.
- Every module should have the good commenting, it helps the developer who comes on the project after you and it also help to maintain the code base.
- One thing you should remember if you are commenting the code which has multiple lines then you should use preprocessors conditional compilation feature (for example, `/*... */`), it increases the code clarity.

Example

// single line

```
#include <stdio.h> // Preprocessor Commands

int main(){ // main function where the program execution begins
    /*
        multi-line
        Comments
    */

    printf("hello world\n"); // single line comments

    return 0; // exit status of main program (could be any number
               // that we agree means the program is finished)
}
```



```
fgets(name, sizeof(name),  
stdin);
```

Thanks

Thank You